CYBERSHIELD

A Project Report

Submitted in partial fulfillment of the

Requirement for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

MR. ADITYA ARVIND KUSHWAHA

Under the esteemed guidance of

Ms. Preeti Nerkar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE

G.K.S. COLLEGE OF ART'S, COMMERCE & SCIENCE

(Affiliated to University of Mumbai)

G.K.S. COLLEGE OF ARTS, COMMERCE & SCIENCE,
KHADAVALI-421 605. MAHARASHTRA

2026-27

G.K.S COLLEGE OF ARTS, COMMERCE AND SCIENCE, KHADAVALI



CERTIFICATE

This is to certify that

ADITYA ARVIND KUSHWAHA

Has successfully completed mid-term project report on

CYBERSHIELD

In partial fulfilment for the T.Y. BSc (Computer Science)

2026-27

UNDER THE GUIDANCE OF

Ms. Preeti Nerkar

Project Guide Examiner

Head of Department Principal

PREFACE

This project serves as a complete source of information about the CyberShield smart threat detection system: how links, files, messages, QR codes, IP addresses and passwords are checked, how the threat score is calculated and how the result is explained to the user. It covers all the activities of “CYBERSHIELD – SMART THREAT DETECTION WEBSITE”. In the system developed by this project, more prominence has been given to a user-friendly environment, privacy of the user’s data and reliable results even when an external service is unavailable.

This project report describes the basic logic used for the software development along with the flow chart, data flow diagrams and UML diagrams so that any reader can easily understand the logic applied. The intrinsic details of every library used have not been included in this report. Although the report has been created with utmost care, there may be some errors. Readers are requested to intimate them to the author.

ACKNOWLEDGEMENT

First and foremost, I offer my sincerest gratitude to all the staff of my college who have supported me throughout my time here, given me valuable knowledge, and moulded and shaped me into the person I am today.

I would like to thank our guide, **PROF. MS. PREETI NERKAR**, for her patience and for sharing her expertise whilst giving me the space to work in my own way.

I would like to thank our Principal **Dr. B. L. JADHAV**, who made available the facilities required for the project.

I cannot thank my family enough for bringing me up the way they did. The source behind my excellence is you.

I would like to thank my friends and seniors for lending me their project experience. This has helped me immensely in developing and testing my web project.

This list would be incomplete without mentioning all of the developers, security researchers and education institutes around the world that share their knowledge, work and wisdom over the Internet, and the providers of the free security APIs used in this project.

ADITYA ARVIND KUSHWAHA

INDEX

Sr. No.	Content	Page No.
1	**Preliminary Investigation**	1
	1.1 Introduction of present system	2
	1.2 Limitation of present system	3
	1.3 Proposed System	4
	1.4 Advantages of proposed system	6
	1.5 System Requirements Specification	7
	1.6 Feasibility study	10
	1.7 Gantt Chart	11
2	**System Analysis**	12
	2.1 Event Table	13
	2.2 Use case Diagram	14
	2.3 ER Diagram	15
	2.4 Activity Diagram	16
	2.5 Class Diagram	18
	2.6 Object Diagram	19
	2.7 Sequence Diagram	20
	2.8 State Diagram	23
	2.9 Collaboration Diagram	24
3	**System Design**	25
	3.1 Component Diagram	26
	3.2 Deployment Diagram	27
	3.3 System Flow Chart	28
	3.4 DFD Level 0, 1, 2	29
4	**System Coding**	34
	4.1 List of Tables	35
	4.2 System Code	38
	4.3 Screen Layout	46
5	**Conclusion and Future work**	49
	5.1 Future Enhancement	50
	5.2 Conclusion	51
6	**References and Bibliography**	52

PRELIMINARY INVESTIGATION

INTRODUCTION OF PRESENT SYSTEM

At present, when a user receives a suspicious link, file or message, there is no single place to check it. The user has to rely on personal judgement, ask friends, search the sender's name online, or open the link and hope that the browser shows a warning. Professional services such as Google Safe Browsing, VirusTotal and AbuseIPDB exist, but each one is a separate website meant for developers and security experts.

Checking a file requires uploading it to a scanner website; checking a message requires reading it carefully for red flags; checking a password usually means typing it into a third-party site, which is itself a security risk. In practice most users do none of these things and simply click, open or pay.

The present system is therefore manual, scattered and dependent on the user's own awareness. CyberShield is developed as an end-user web application that brings these checks together in one place.

LIMITATIONS OF PRESENT SYSTEM

- Users must know about and visit several different security websites to check one item fully.
- Existing tools return raw technical data (engine names, categories, hashes) instead of a simple verdict and advice.
- Browser warnings appear only after the link has already been clicked.
- Scam SMS and WhatsApp messages written in Hinglish are not understood by keyword-based filters.
- Checking a password on an ordinary website means sending the password to that website.
- There is no awareness material built into the tools, so users keep falling for the same KYC, OTP and UPI frauds.
- All these things cannot be achieved in the existing system.

PROPOSED SYSTEM

- In order to avoid the limitations of the existing system, CyberShield is developed as a web-based smart threat detection application.
- Links are checked against Google Safe Browsing, more than seventy VirusTotal engines, domain age (RDAP) and phishing heuristics in one request.
- Files are identified by a SHA-256 hash computed in the browser and looked up on VirusTotal before any upload; files are never stored on the server.
- SMS and email messages are analysed by Gemini AI together with a rule engine for Indian scam patterns in English and Hinglish, and every link inside is scanned.
- QR codes are decoded in the browser; UPI payment requests are parsed and the user is warned that nobody needs to scan a QR code to receive money.
- IP addresses are checked against AbuseIPDB and geolocation data; passwords are checked against Have I Been Pwned using k-anonymity so the password never leaves the browser.
- All signals are converted into a weighted threat score (0–100), a verdict of Safe, Suspicious or Dangerous, a plain-language explanation and a downloadable PDF report.

FEATURES OF PROPOSED SYSTEM

The functionalities provided by the project are as follows.

- Link Scanner with Safe Browsing, VirusTotal, domain age and heuristics
- File Scanner with browser-side hashing and hash-first VirusTotal lookup
- SMS / Email Scam Checker with AI analysis and highlighted red flags
- QR Code Scanner with camera, image upload and UPI parsing
- IP Address Checker with abuse reputation and geolocation
- Password Leak Checker with k-anonymity and a strength meter
- Threat score gauge, verdict badge, score breakdown and per-source findings
- PDF report download and "Check again" for pending VirusTotal analyses
- Scam awareness page with seven Indian scam types, helplines and a ten-question quiz
- Live public scan counter and secure, rate-limited API

The application runs in any modern browser on a computer or mobile phone and does not require registration, which makes it immediately useful for ordinary users.

ADVANTAGES OF PROPOSED SYSTEM

The various advantages of the proposed system are:

ONE-STOP CHECKING

Six different kinds of input are checked from one page. The user does not need to know which security service answers which question.

SPEED

All sources are queried in parallel and known results are cached for one hour, so most scans finish in a few seconds. Repeated scans of the same link return instantly.

ACCURACY

Because the verdict combines several independent sources, a single wrong signal cannot produce a Dangerous verdict by itself, which reduces false alarms while still catching real threats.

PRIVACY AND SAFETY OF DATA

Files are identified by hash and uploaded only when unknown, passwords never leave the browser, no account is needed and no personal data is stored. API keys stay on the server.

EASY TO UNDERSTAND

Every result has a 0–100 score, a colour-coded verdict, a plain-language explanation, a list of reasons and practical advice such as the 1930 cyber-fraud helpline.

RELIABILITY

If one service fails or its free quota is exhausted, the others continue and the report clearly marks what could not be checked instead of showing a false Safe.

SYSTEM REQUIREMENT SPECIFICATION

HARDWARE REQUIREMENT

The application runs in a web browser, so it does not require special hardware. The suggested configuration for development and testing is:

- Intel Core i3 / Apple M1 or better processor
- 8 GB of memory
- 256 GB of disk space
- 15” screen or larger

Realistically the website can be used on much more modest hardware, including mobile phones. The minimum needed for adequate performance is:

- Any dual-core processor
- 4 GB of memory
- 13” screen or a mobile phone
- 1 GB of free disk space for a local setup

And the components that are not variable are:

- Keyboard and mouse (or touch screen)
- Internet connection (required for the security APIs)
- Webcam or phone camera (optional, for scanning QR codes)

SOFTWARE REQUIREMENT

• Operating System	:	macOS / Windows / Linux
• Front End	:	React 19, Vite, Tailwind CSS v4, Framer Motion
• Back End	:	Node.js 20 with Express 5
• Database	:	Cloud Firestore (public scan counter)
• Libraries	:	jsQR, jsPDF, Axios, Multer, express-rate-limit, @google/genai
• Security APIs	:	Google Safe Browsing v4, VirusTotal v3, AbuseIPDB v2, RDAP, ip-api, Have I Been Pwned
• IDE	:	Visual Studio Code
• Browser	:	Google Chrome
• Hosting	:	GitHub Pages (client) and Render (API)

INTRODUCTION

To develop a feature-rich web solution that lets any Internet user in India check suspicious links, files, messages, QR codes, IP addresses and passwords before they click, open, scan or pay.

1. Abstract:

The ‘CyberShield’ application keeps track of the information needed to judge a threat: the verdict of Google Safe Browsing, the detections of more than seventy antivirus engines on VirusTotal, the registration age of a domain, the abuse reports of an IP address, the breach history of a password and the scam patterns found in a message. It converts this information into a single threat score and explains it in plain language.

The users of the system are ordinary citizens who receive suspicious content through SMS, WhatsApp, email and social media. No login is required. The administrator of the system is the developer, who configures the API keys on the server and deploys the client and the API.

When a link is submitted, the system normalises it, runs heuristics, queries the security sources in parallel and caches the result. When a file is submitted, its SHA-256 hash is computed in the browser and looked up first; only unknown files are uploaded and they are never stored. When a message is submitted, Gemini AI and a rule engine analyse it and every link inside is scanned. QR codes are decoded in the browser and UPI requests are explained. Passwords are checked with k-anonymity entirely in the browser.

The result page shows an animated threat gauge, a verdict badge, the explanation, a breakdown of the score, one card per source, the antivirus engine grid and advice, and the user can download a PDF report. An awareness section explains the most common Indian scams and offers a quiz.

FEASIBILITY STUDY

Feasibility study is conducted once the problem is clearly understood. The objective is to determine quickly and at a minimum expense how to solve the problem. It involves details of the system and its operations. The system has been tested for feasibility on the following points:

• TECHNICAL FEASIBILITY:

It is an evaluation of hardware and software and how they meet the needs of the proposed system. It is possible to develop the system using React and Tailwind CSS as front-end tools and Node.js with Express as back-end tools. All the security services used offer free developer tiers and well-documented REST APIs, so the system can be built without any special infrastructure.

• ECONOMICAL FEASIBILITY:

The software uses only free tiers of the security APIs, free hosting on GitHub Pages and Render, and open-source libraries. It saves users from financial loss due to fraud and saves time compared with checking several websites manually, so the benefit far exceeds the cost.

• OPERATIONAL FEASIBILITY:

The application needs no installation or registration; it works in any browser and on mobile phones. Results are explained in simple language with advice, so no training is required to use it.

• FINANCIAL FEASIBILITY:

The total cost of the project is estimated and it was found that the software can be developed and hosted within the free limits of the services used, so it can easily be afforded by a student or a college.

GANTT CHART

Project Steps	Jul 26	Aug 26	Sep 26	Oct 26	Nov 26	Dec 26	Jan 27
System study and analysis							
Problem definition							
Requirements gathering							
Diagrams and design							
Coding (client + API)							
Implementation 1 (scanners)							
Testing 1							
Review 1.1							
Implementation 2 (QR, password, PDF)							
Testing 2.1							
Deployment and report							

Legend: shaded cells show the planned duration of each step. The project was carried out from July 2026 to January 2027.

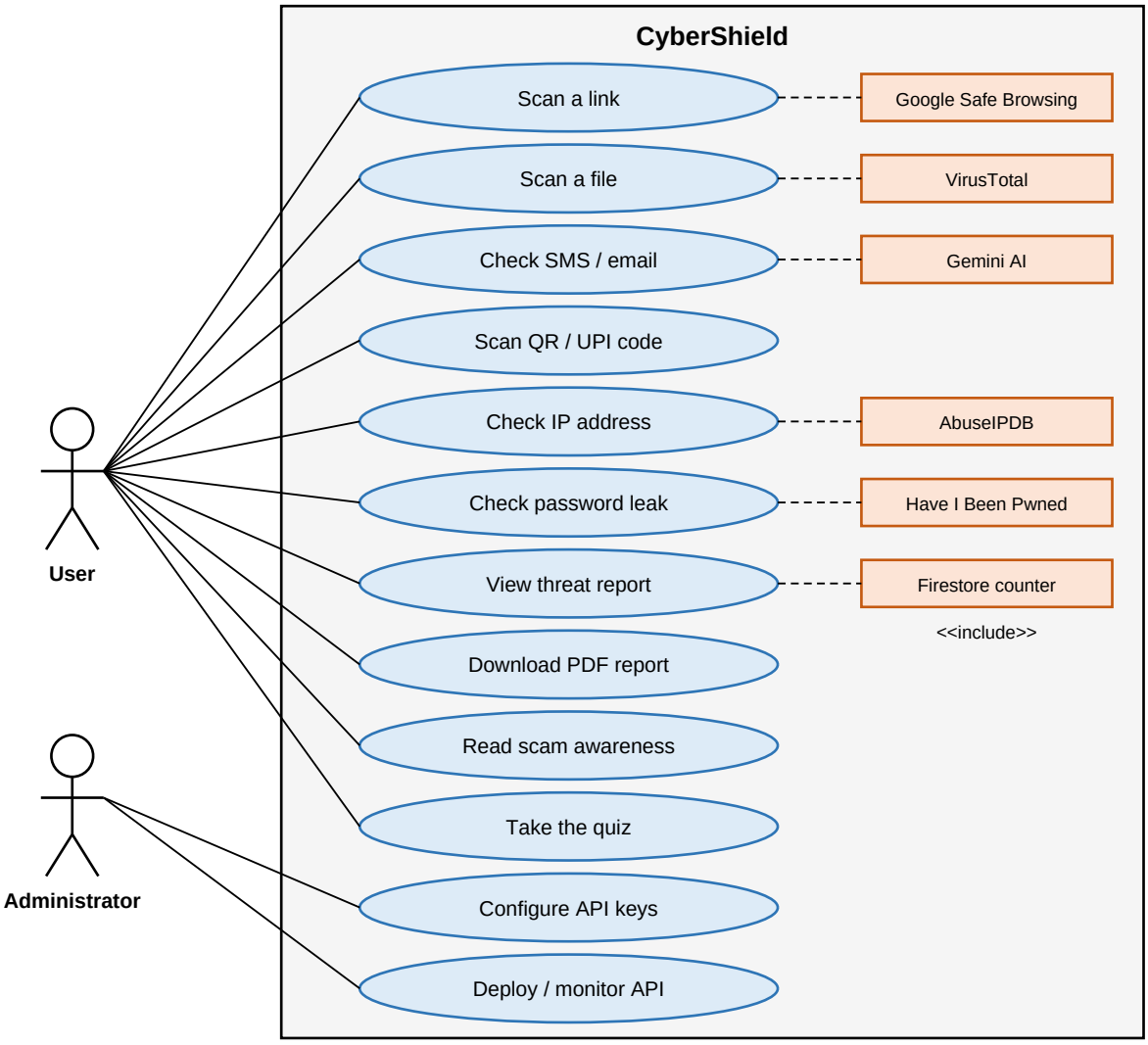
SYSTEM ANALYSIS

EVENT TABLE

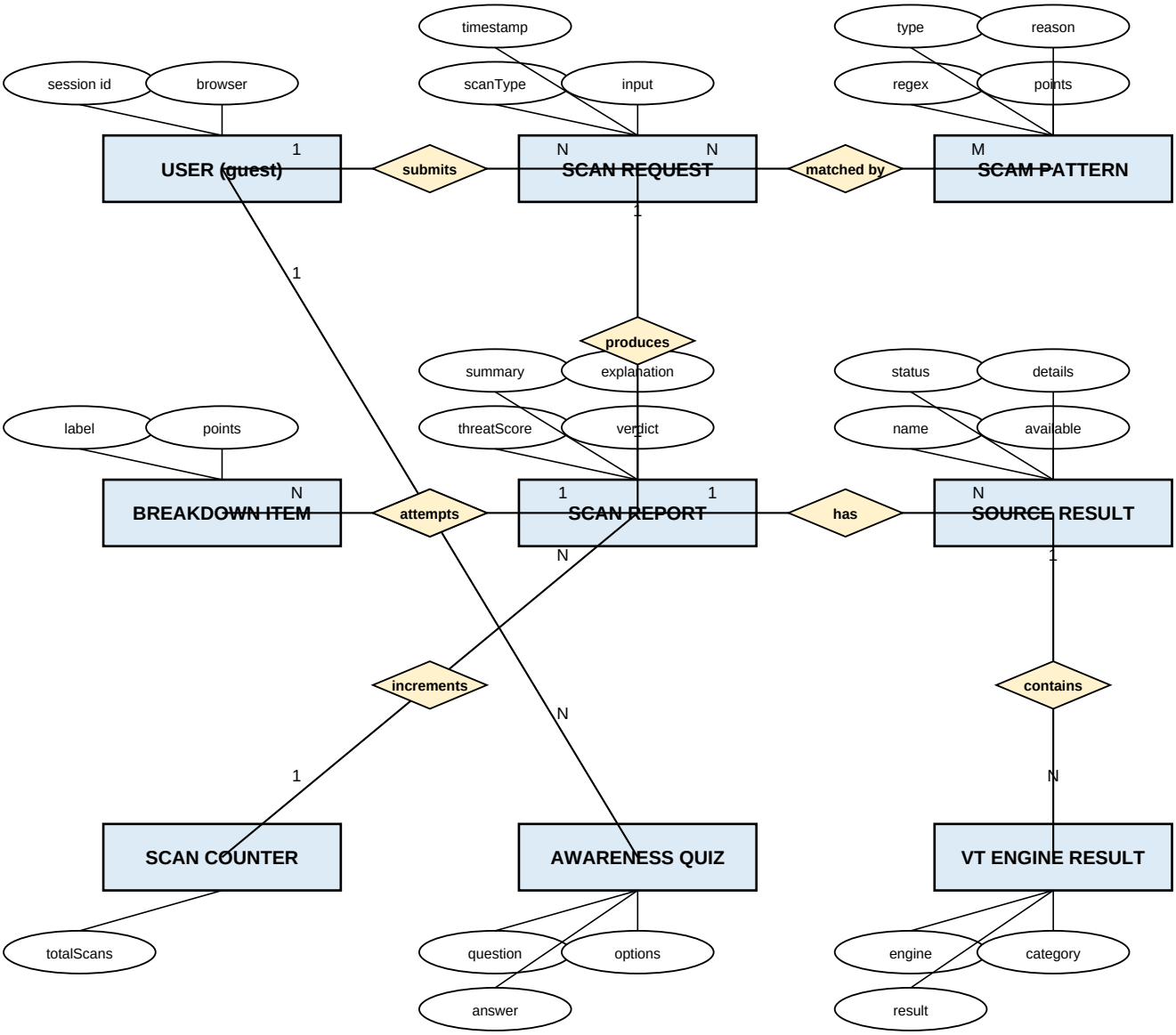
Event	Trigger	Source	Activity	Response	Destination
Checking a link	User pastes a link and clicks Scan	User	Validate, query Safe Browsing, VirusTotal, RDAP, heuristics	Threat score, verdict, findings	User
Checking a file	User chooses a file	User	Hash in browser, look up hash, upload if unknown	Detection count, malware family	User
Checking a message	User pastes SMS / email text	User	Gemini analysis, scam rules, scan links inside	Scam type, red flags, advice	User
Scanning a QR code	Camera frame or image decoded	User	Classify content, parse UPI, scan link	Payment warning or link report	User
Checking an IP	User enters an address	User	Validate, query AbuseIPDB and geolocation	Abuse confidence, ISP, country	User
Checking a password	User types a password	User	SHA-1 locally, k-anonymity lookup, strength meter	Breach count, strength	User
Downloading report	User clicks Download PDF	User	Generate PDF with jsPDF	PDF file	User
Counting scans	Any scan completes	Client	Increment Firestore counter	Live total on Home page	All users
Taking the quiz	User answers a question	User	Check answer, show explanation	Score and grade	User

USE CASE DIAGRAM

Use case Diagram for CyberShield

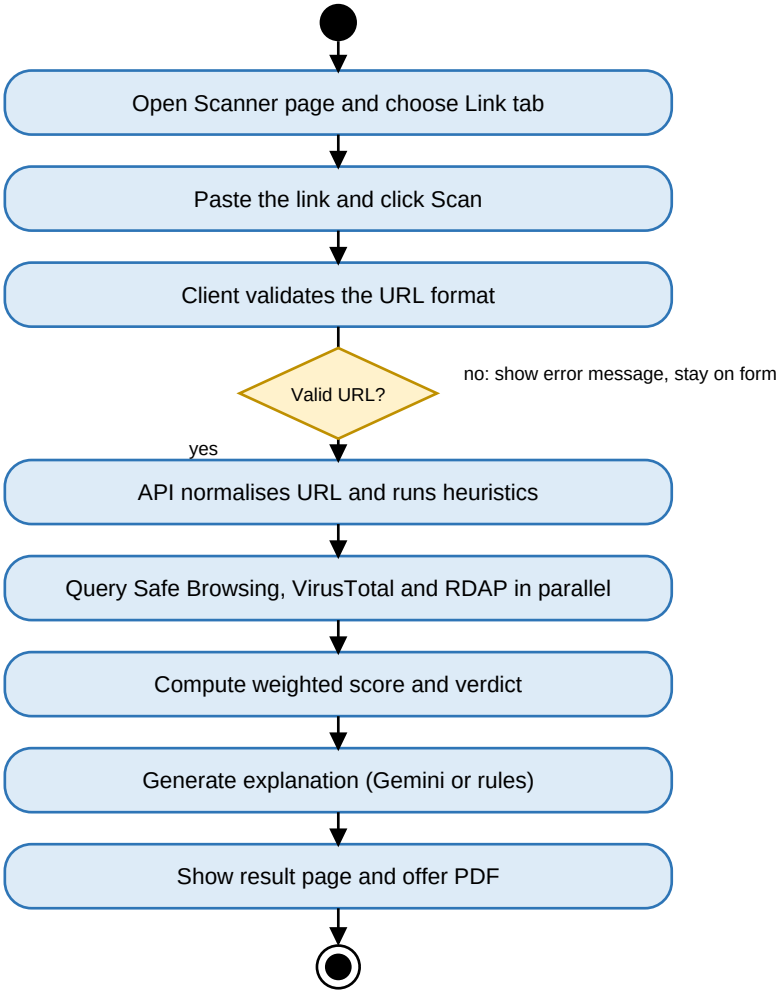


ER DIAGRAM

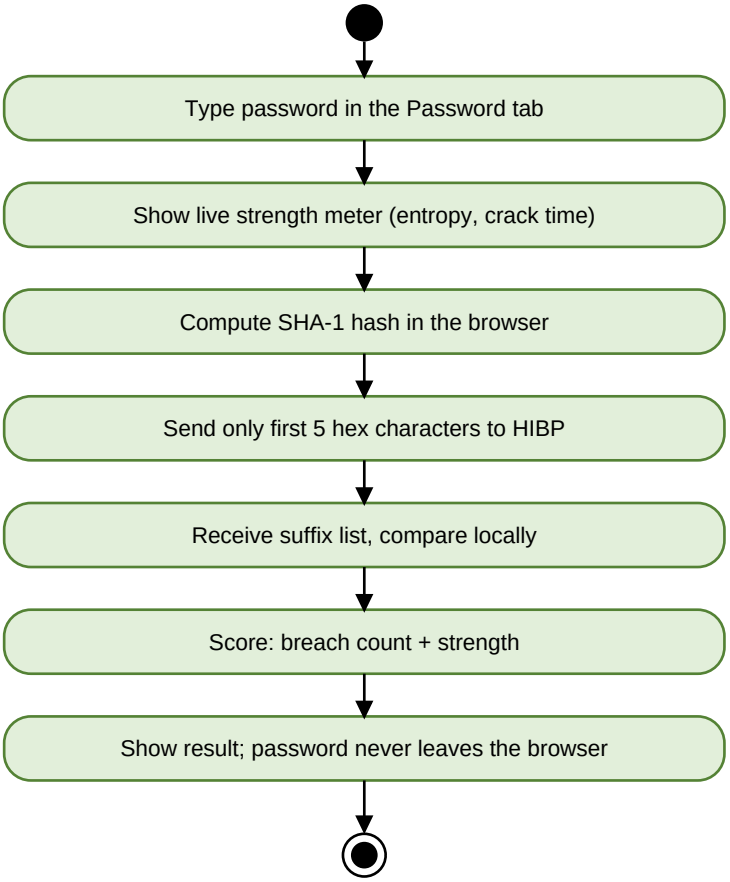

timestamp
type
reason
session id
browser
scanType
input
regex
points
USER (guest)
submits
SCAN REQUEST
matched by
SCAM PATTERN
1
N
N
M
1
1
produces
summary
explanation
status
details
label
points
threatScore
verdict
name
available
BREAKDOWN ITEM
attempts
SCAN REPORT
has
SOURCE RESULT
N
1
1
N
N
1
increments
contains
N
1
N
SCAN COUNTER
AWARENESS QUIZ
VT ENGINE RESULT
totalScans
question
options
engine
category
answer
result

ACTIVITY DIAGRAMS:

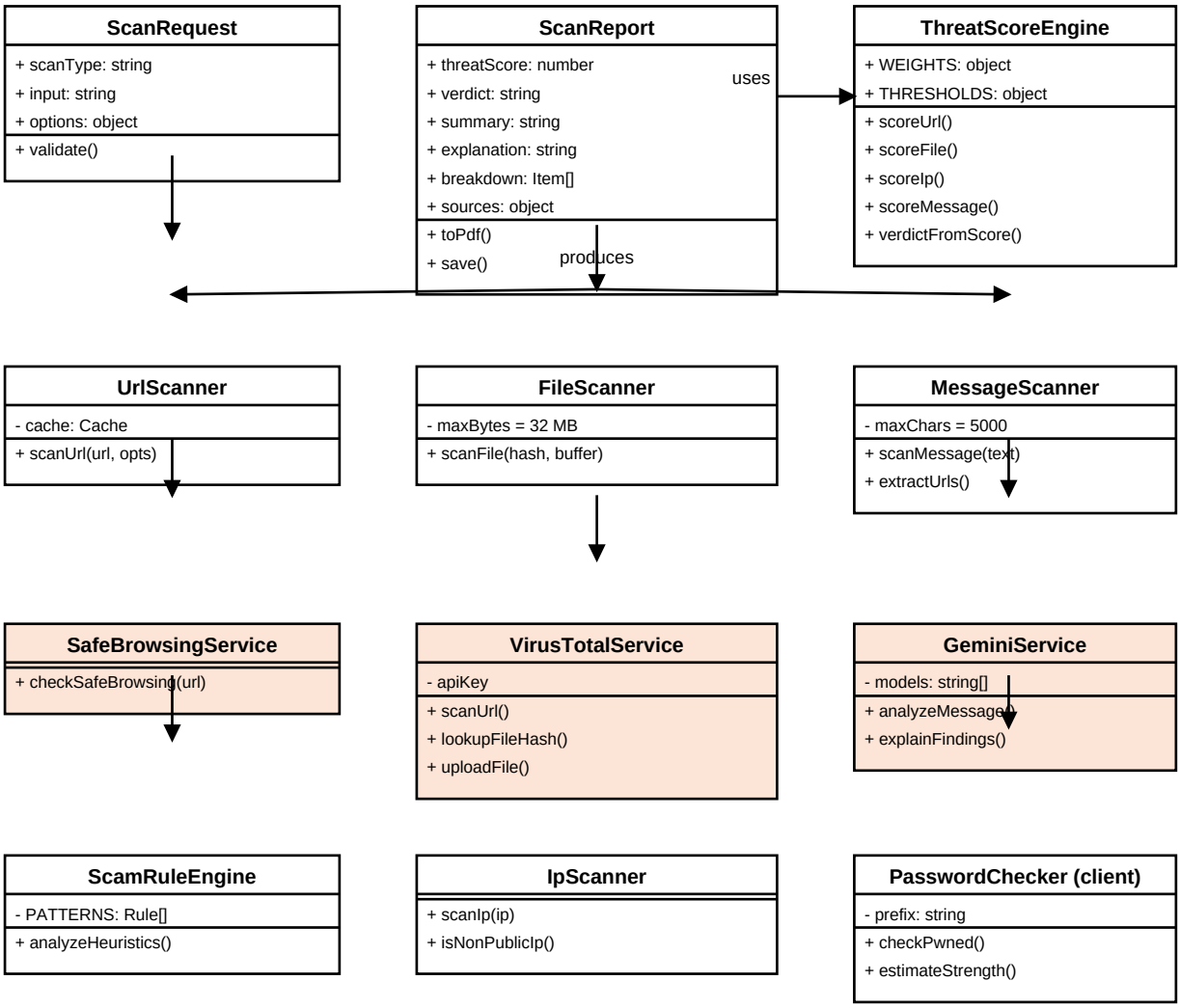
Link Scan Activity



Password Check Activity:

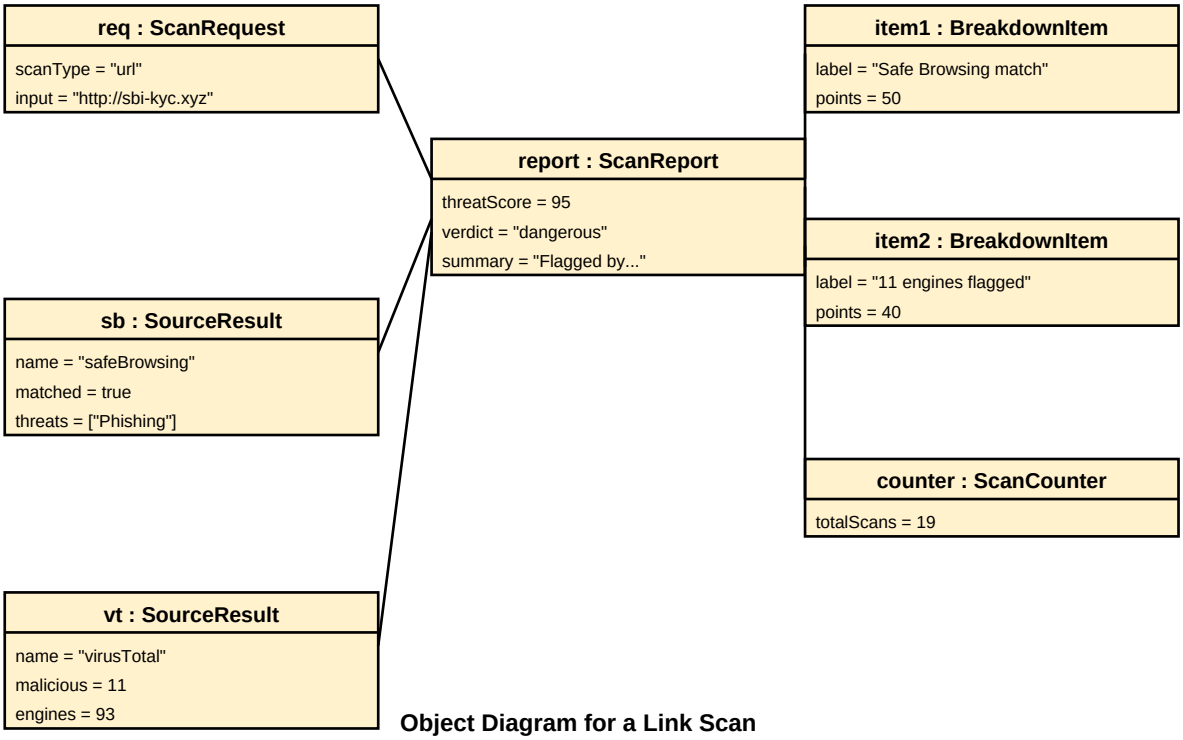


CLASS DIAGRAM



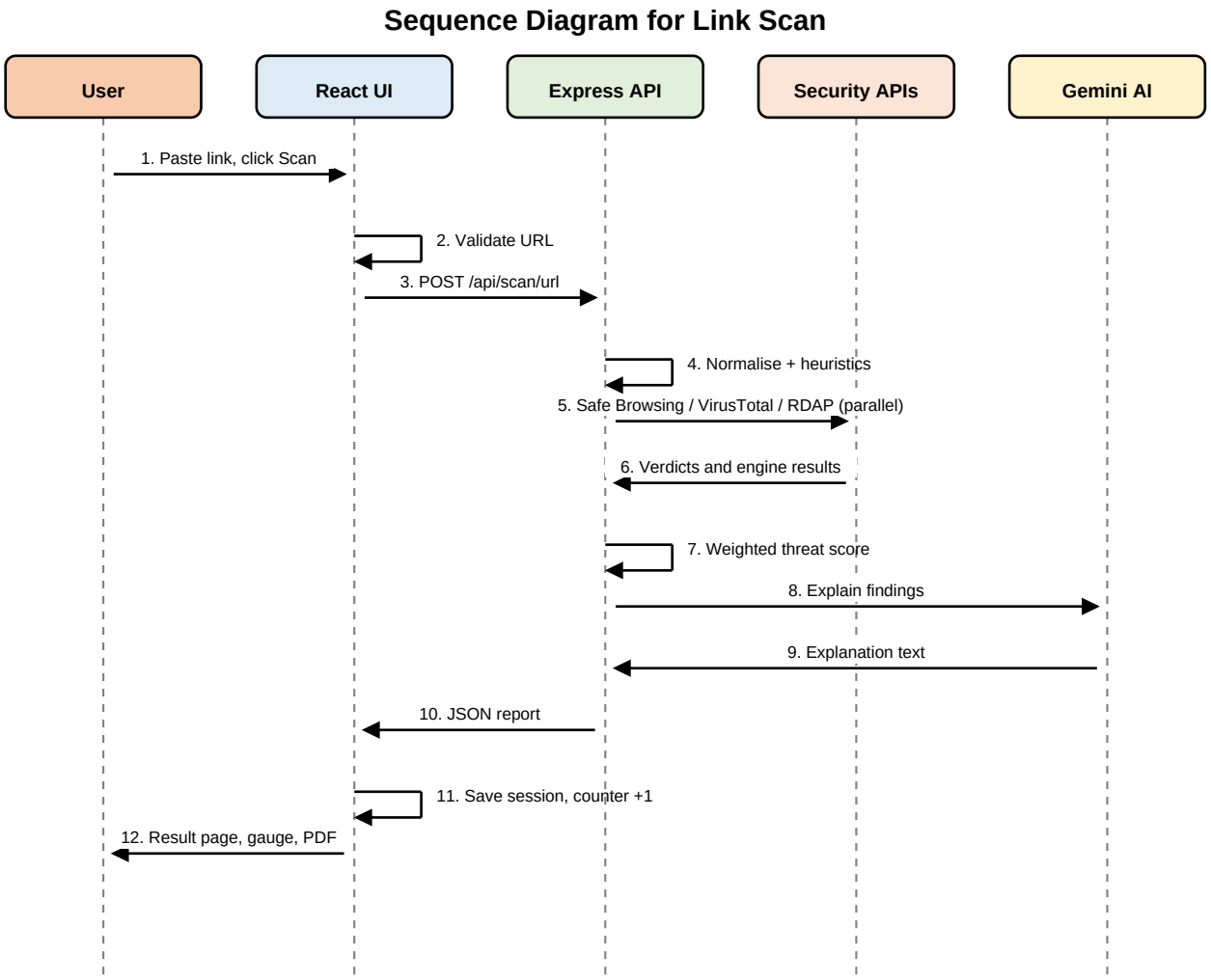
Class Diagram for CyberShield

OBJECT DIAGRAM

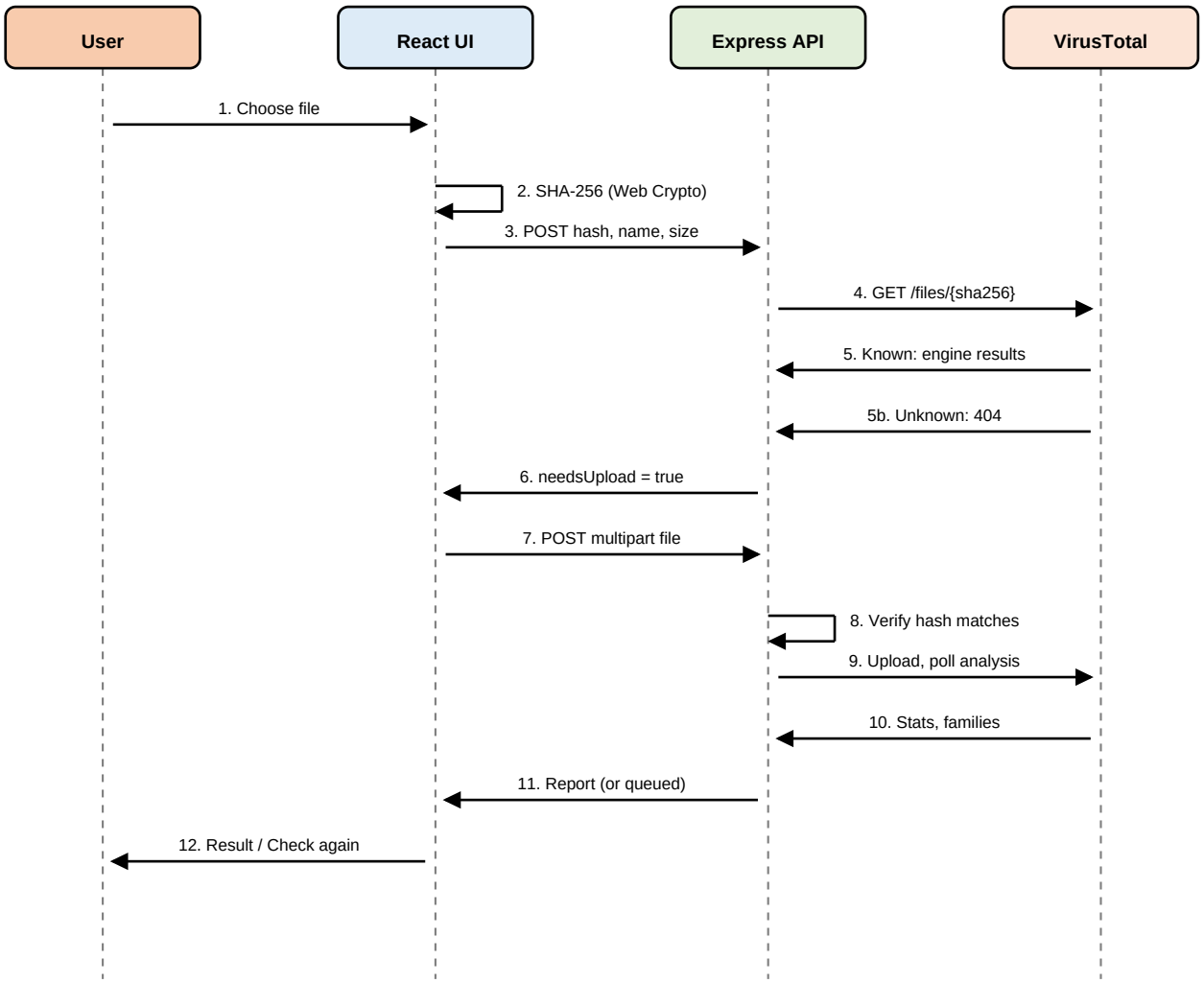


Object Diagram for a Link Scan

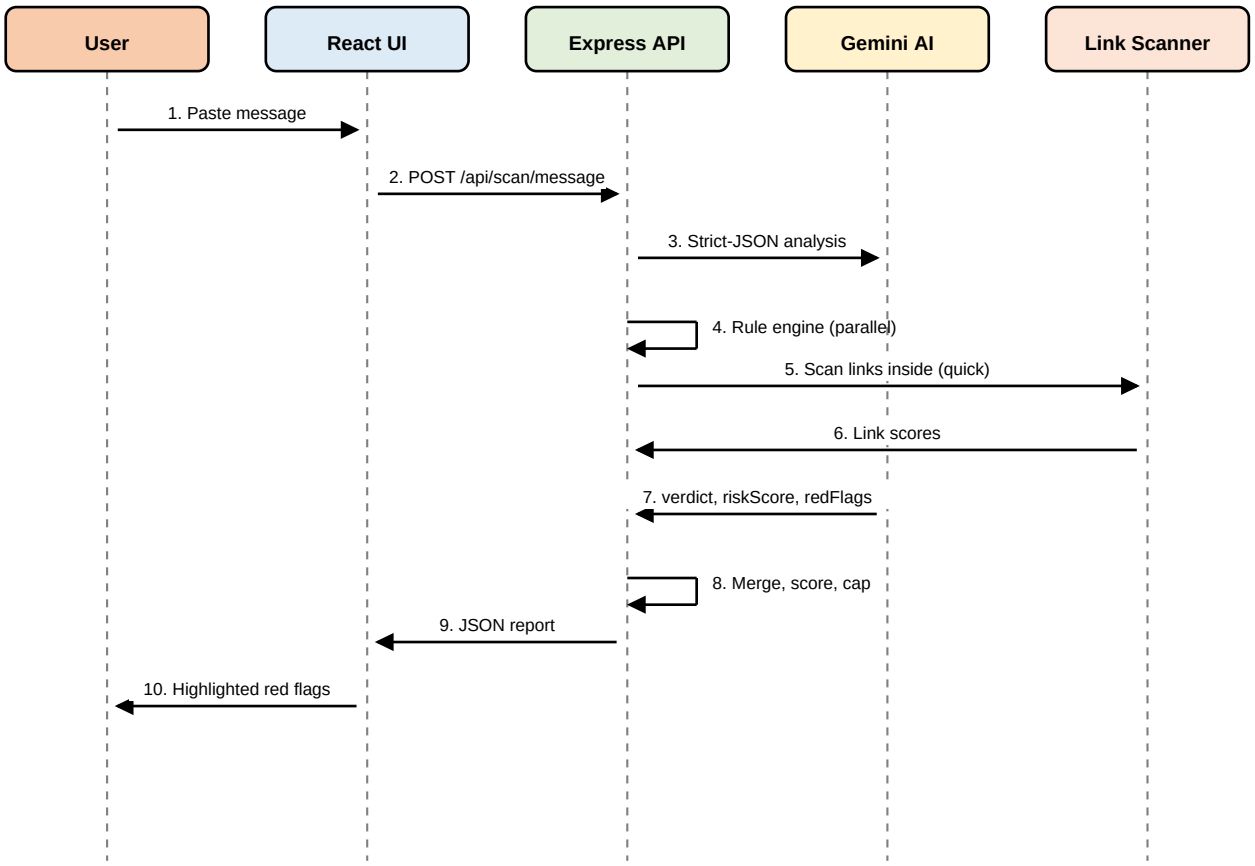
SEQUENCE DIAGRAM



Sequence Diagram for File Scan

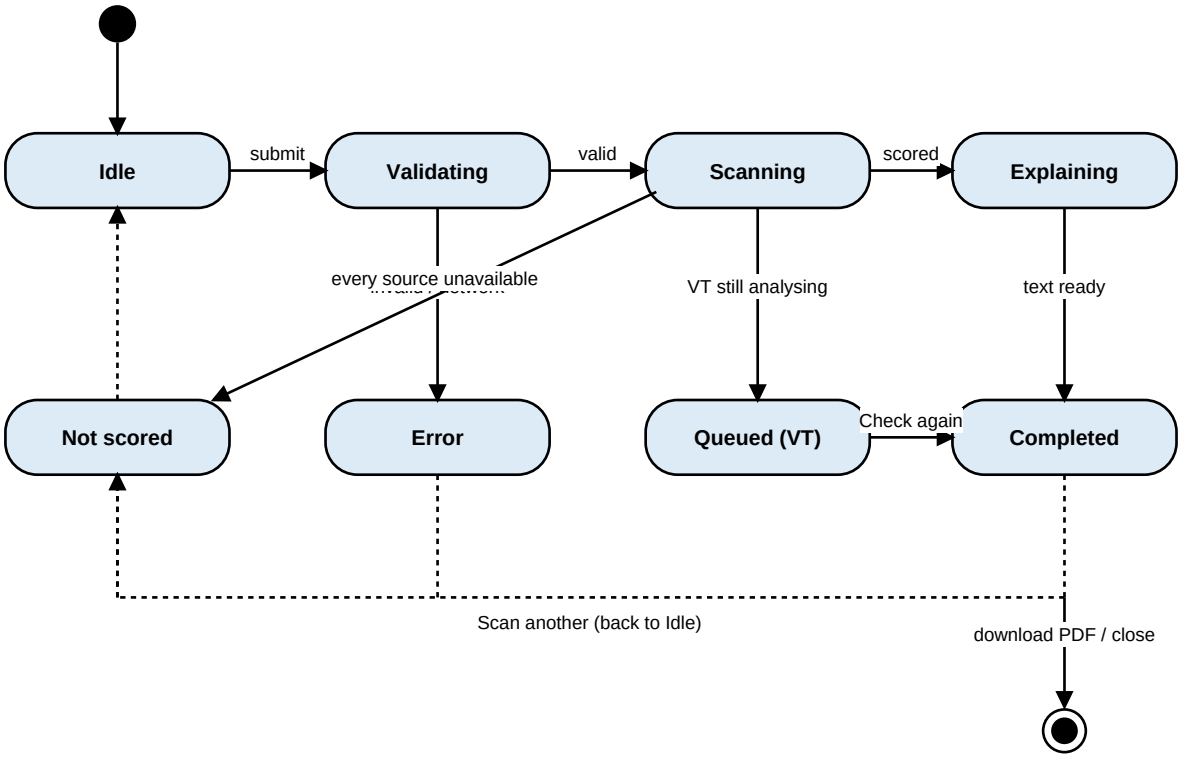


Sequence Diagram for Message Scan



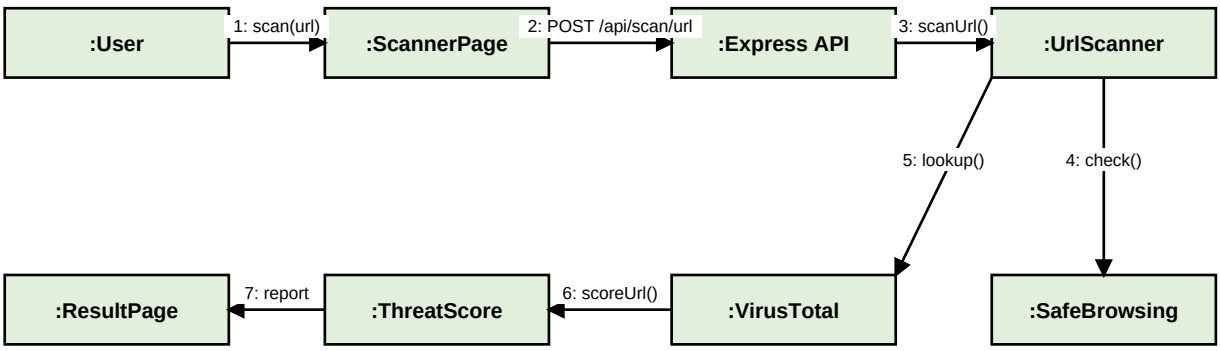
STATE DIAGRAM

State Diagram for a Scan



COLLABORATION DIAGRAM

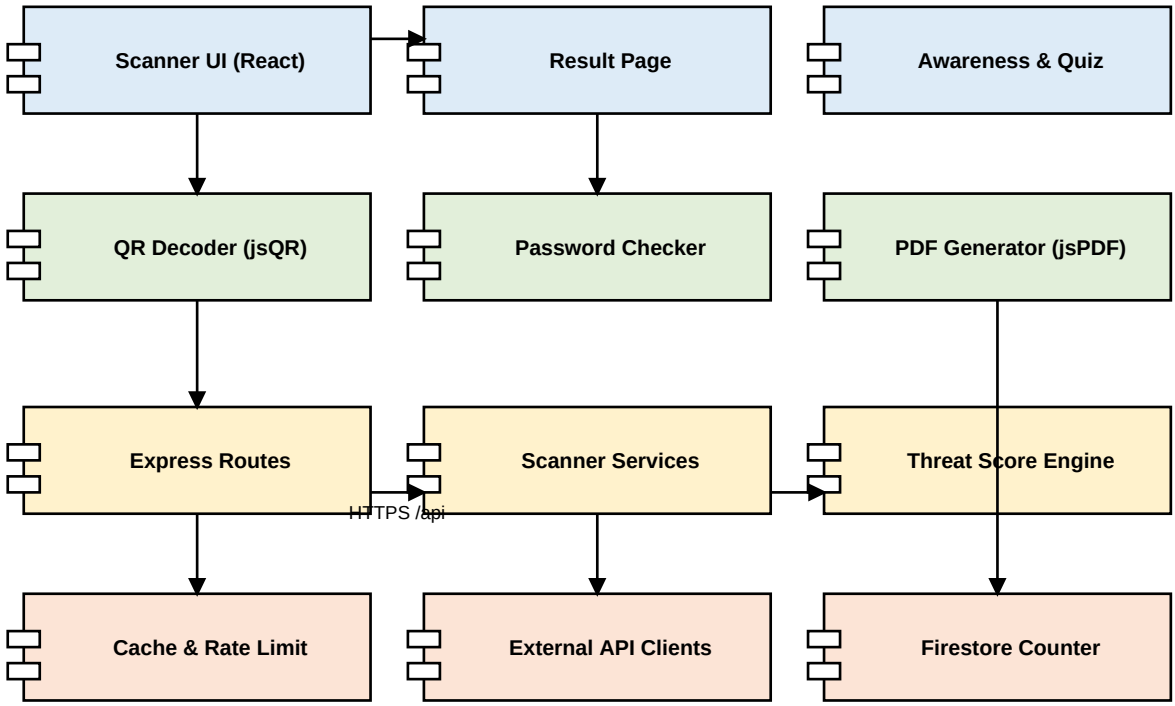
Collaboration Diagram for a Link Scan



SYSTEM DESIGN

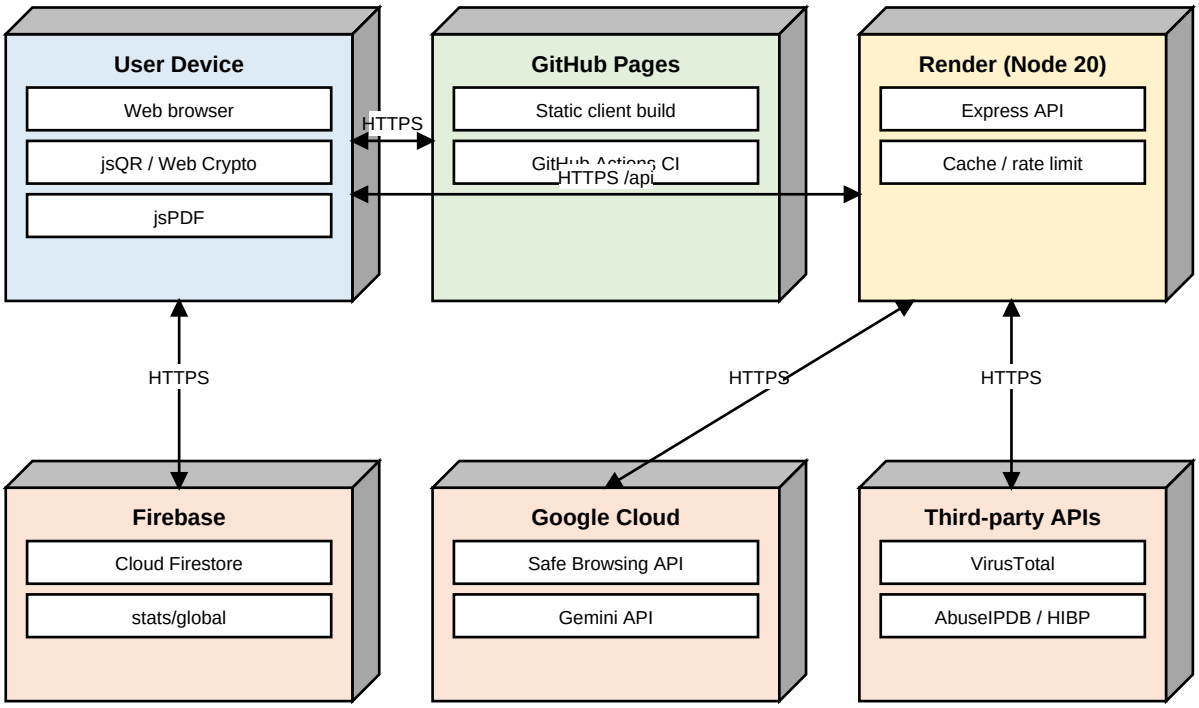
COMPONENT DIAGRAM

Component Diagram for CyberShield



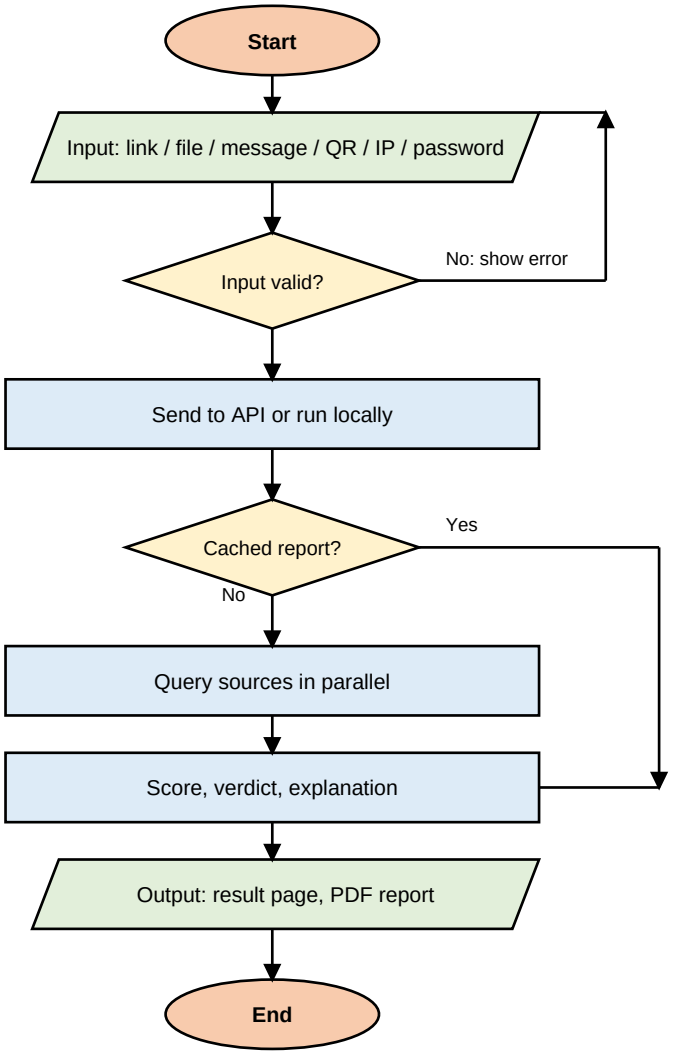
DEPLOYMENT DIAGRAM

Deployment Diagram for CyberShield



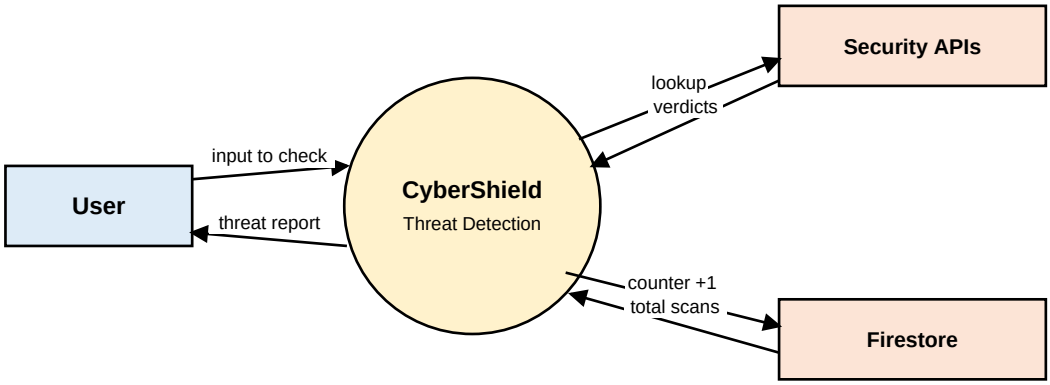
SYSTEM FLOW CHART

System Flow Chart

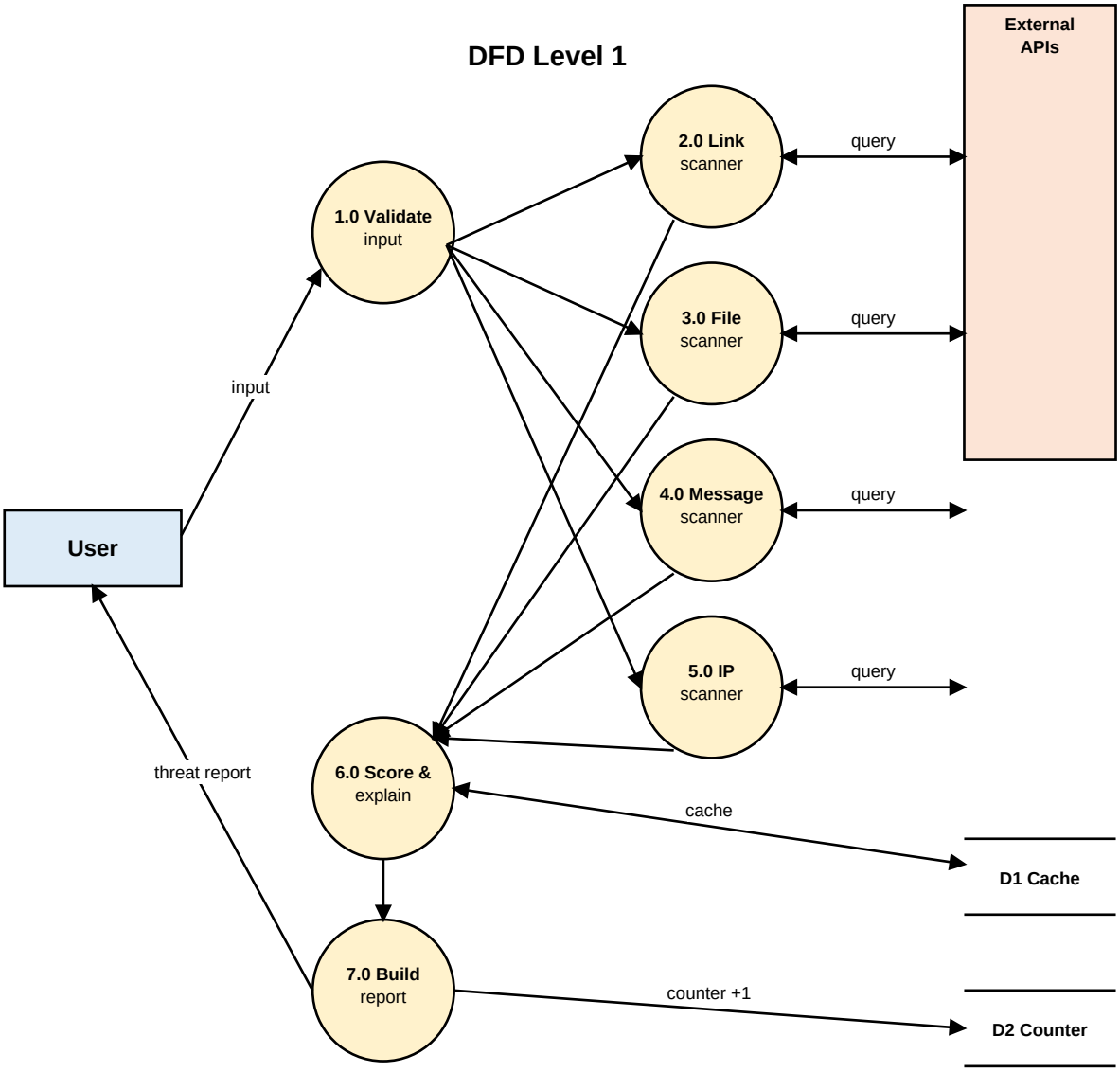


DFD LEVEL 0, 1, 2

DFD Level 0 (Context Diagram)

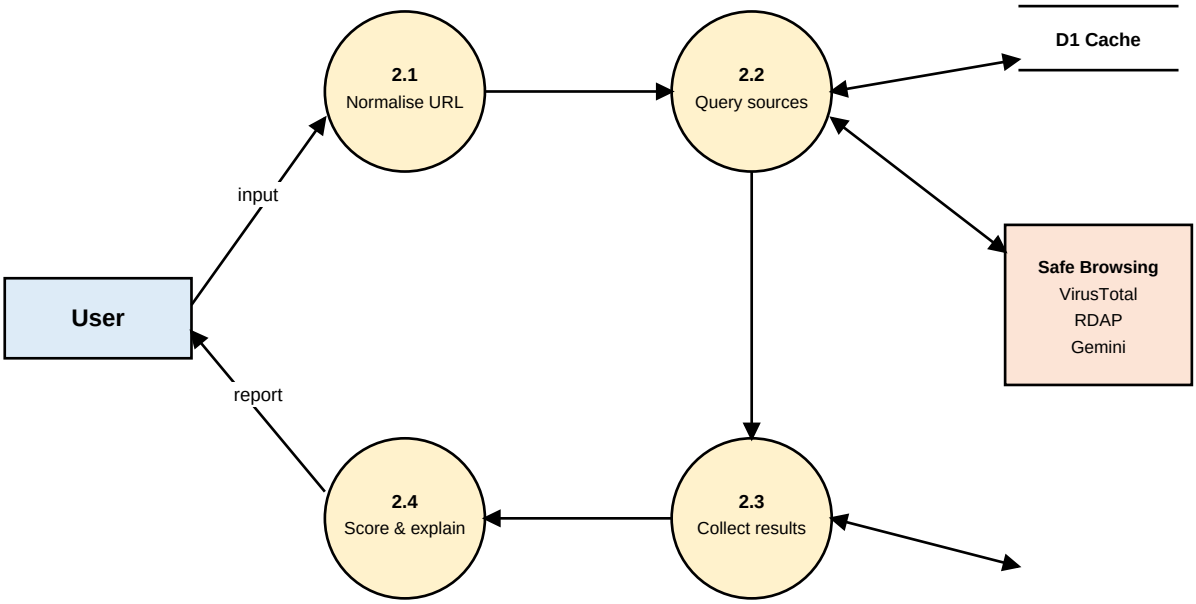


DFD Level 0 shows the whole system as one process exchanging data with the user, the external security APIs and the Firestore counter.

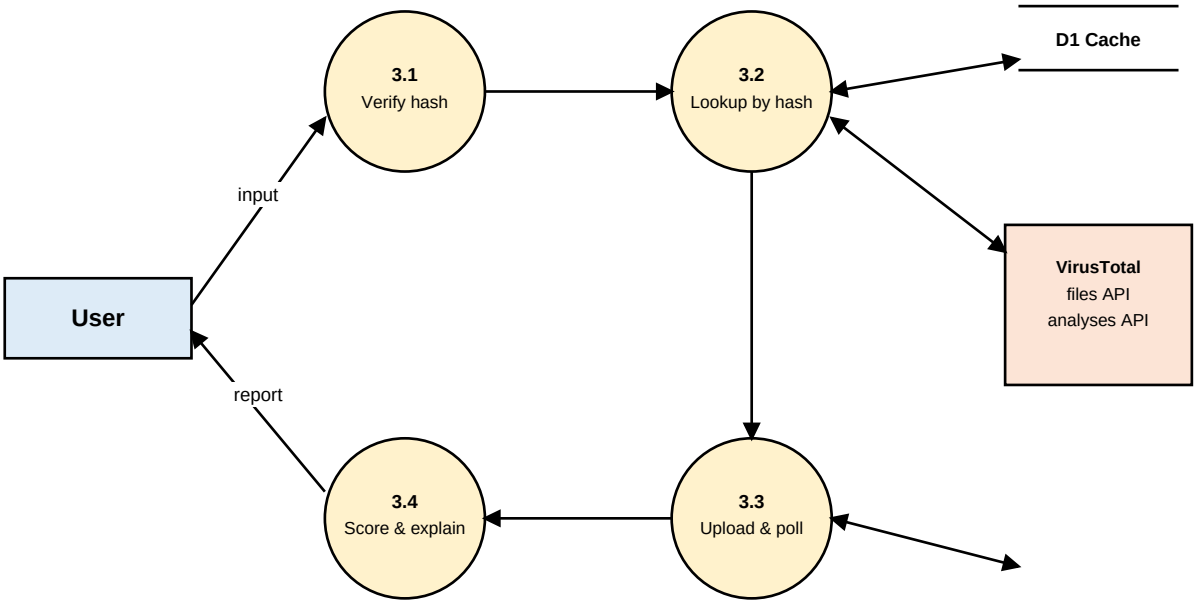


DFD Level 1 breaks the system into validation, the four server-side scanners, scoring and report building, with the cache and counter as data stores.

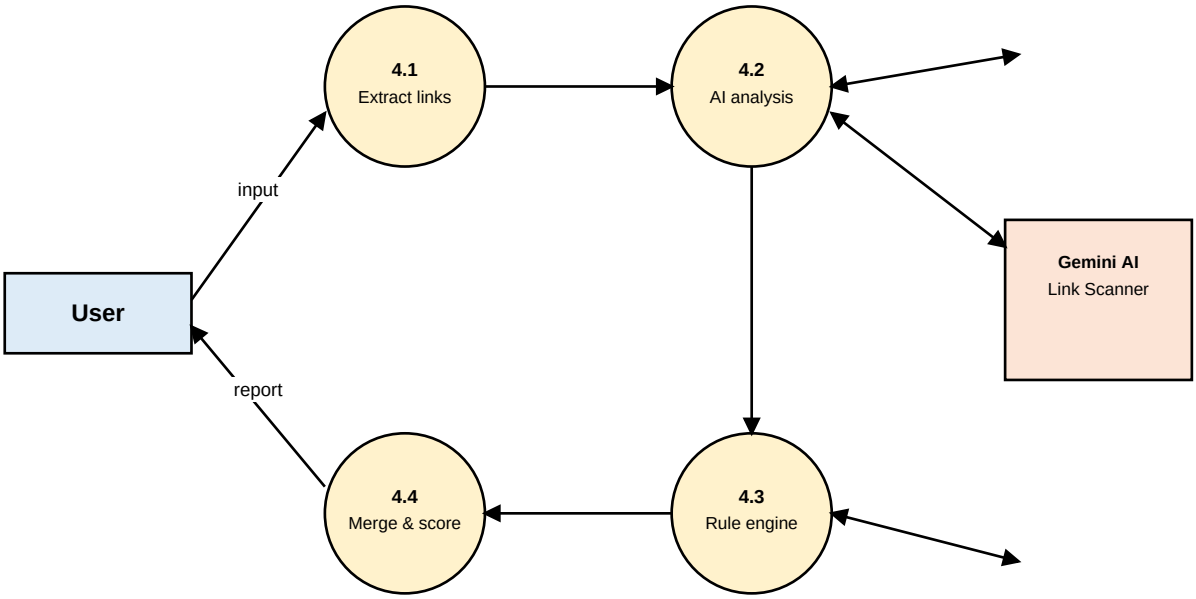
DFD Level 2 for the Link Scanner



DFD Level 2 for the File Scanner



DFD Level 2 for the Message Scanner



SYSTEM CODING

List of Tables

CyberShield does not use a relational database: scan reports are JSON documents exchanged between the client and the API, and only one document is stored permanently in Cloud Firestore. The following tables describe the data structures used by the system.

Table 1: Scan Report (JSON returned by every scanner)

Field	Type	Description
scanType	string	url, file, message, ip, qr or password
input / inputPreview	string	The scanned item and a shortened display version
threatScore	number / null	0–100, or null when nothing could be scored
verdict	string	safe, suspicious, dangerous or unknown
summary	string	One-line result
explanation	string	Plain-language explanation (AI or rule-based)
aiAvailable	boolean	Whether Gemini produced the explanation
breakdown[]	array	Items of {label, points} that add up to the score
sources	object	Per-source findings (see tables below)
unavailableSources[]	array	Sources that could not be checked
scannedAt	ISO date	Time of the scan
cached	boolean	Served from the one-hour cache

Table 2: Safe Browsing Source Result

Field	Type	Description
available	boolean	Service answered
matched	boolean	URL found on a threat list
threats[]	array	Malware, Phishing, Unwanted software, Harmful application

Table 3: VirusTotal Source Result

Field	Type	Description
available / status	boolean / string	completed, queued, rate_limited, not_configured
stats	object	malicious, suspicious, harmless, undetected, timeout counts
engines[]	array	Per-engine {name, category, result}
permalink	string	Link to the full VirusTotal report
fileType / families[]	string / array	File type and malware family names (files only)

Table 4: Domain Age (RDAP) and Heuristics

Field	Type	Description
domain	string	Registrable domain
registered / ageDays	date / number	Registration date and age in days
isNew	boolean	Younger than 30 days
isHttps	boolean	Uses HTTPS
keywords[]	array	Suspicious words found in the URL
isIpUrl / excessiveSubdomains	boolean	Raw IP host, 3+ subdomains
hasAtSymbol / hasPunycode	boolean	"@" trick, look-alike domain
flags[]	array	{severity, label, detail} shown on the result page

Table 5: Message Analysis Result

Field	Type	Description
source	string	gemini or rules
verdict	string	safe, suspicious or scam
riskScore	number	0–100 AI or rule risk
scamType	string	KYC fraud, OTP scam, Fake job offer, Digital arrest, etc.
redFlags[]	array	{phrase, reason} highlighted in the message
explanation / advice	string	AI text
links[]	array	Embedded links with their own threat scores

Table 6: IP Reputation Result

Field	Type	Description
abuseConfidenceScore	number	0–100 from AbuseIPDB
totalReports / numDistinctUsers	number	Reports in the last 90 days
countryCode / isp / usageType	string	Network ownership details
isTor / isWhitelisted	boolean	Tor exit node, whitelisted
geo: country, city, asn, isProxy, isHosting	mixed	Geolocation from ip-api

Table 7: Password Check Result (client only)

Field	Type	Description
prefix	string	First 5 characters of the SHA-1 hash that were sent
breachCount	number	Times seen in breaches
candidates	number	Suffixes returned for the prefix
strength.score / label	number / string	0–4, Very weak to Very strong
entropyBits / crackTime	number / string	Local strength estimate
warnings[] / suggestions[]	array	Tips shown to the user

Table 8: UPI QR Result (client only)

Field	Type	Description
kind	string	url, upi, wifi, tel, contact, text
payeeAddress / payeeName	string	UPI ID and name
amount / currency	number / string	Pre-filled amount, if any
merchantCode / txnRef	string	Merchant code and transaction reference

Table 9: Firestore Document stats/global

Field	Type	Description
totalScans	number	Public counter; rules allow only +1 per write

Table 10: Threat Score Weights (server/utils/threatScore.js)

Field	Type	Description
SAFE_BROWSING_MATCH	50	Google flags the URL
VT_PER_ENGINE / VT_MAX	10 / 40	Per VirusTotal engine, capped
YOUNG_DOMAIN	15	Domain younger than 30 days
NO_HTTPS	5	HTTP instead of HTTPS
SUSPICIOUS_KEYWORDS	10	kyc, verify, login, otp, bank names
IP_ADDRESS_URL / EXCESSIVE_SUBDOMAINS	10 / 5	Structural tricks
AT_SYMBOL_OR_PUNYCODE	10	"@" trick or look-alike domain
Thresholds	30 / 60	Safe up to 30, Suspicious up to 60, Dangerous above

SYSTEM CODE

Express application (server/index.js):

```
import express from 'express';
import cors from 'cors';
import scanRouter from './routes/scan.js';
import { scanLimiter } from './middleware/rateLimiter.js';
import { errorHandler, notFound, httpError } from './middleware/errorHandler.js';
import { allowedClientOrigins } from './utils/origins.js';

const app = express();
const PORT = Number(process.env.PORT) || 5001;
app.set('trust proxy', 1);
const allowedOrigins = allowedClientOrigins(process.env.CLIENT_ORIGIN);
app.use(cors({
  origin(origin, cb) {
    if (!origin || allowedOrigins.includes(origin)) return cb(null, true);
    return cb(httpError(403, 'forbidden_origin', 'This browser origin is not allowed.'
));
  },
  methods: ['GET', 'POST'],
}));
app.use(express.json({ limit: '64kb' }));
app.get('/api/health', (_req, res) => res.json({ ok: true, apis: configStatus() }));
app.use('/api/scan', scanLimiter, scanRouter);
app.use(notFound);
app.use(errorHandler);
app.listen(PORT, () => console.log(`CyberShield API listening on ${PORT}`));
```

Scan routes (server/routes/scan.js):

```
router.post('/url', async (req, res) => {
  const { url } = req.body || {};
  if (typeof url !== 'string' || !url.trim()) throw httpError(400, 'missing_url', 'Ple
ase provide a link to scan.');
  res.json(await scanUrl(url, { skipCache: req.query.fresh === '1' }));
});

router.post('/file', upload.single('file'), uploadErrorHandler, async (req, res) => {
  const body = req.body || {};
  const report = await scanFile({
    sha256: body.sha256,
    fileName: body.fileName || req.file?.originalname,
    size: Number(body.size) || req.file?.size,
    mimeType: body.mimeType || req.file?.mimetype,
    buffer: req.file?.buffer || null,
  });
  if (req.file) req.file.buffer = null;   // never keep the bytes
  res.json(report);
});

router.post('/message', async (req, res) => {
  const { text } = req.body || {};
  if (typeof text !== 'string') throw httpError(400, 'missing_text', 'Paste the SMS or
 email text to check.');
  res.json(await scanMessage(text));
});

router.get('/ip', async (req, res) => res.json(await scanIp(req.query.ip)));
```

Threat score engine (server/utils/threatScore.js):

```
export const WEIGHTS = {
  url: { SAFE_BROWSING_MATCH: 50, VT_PER_ENGINE: 10, VT_MAX: 40, YOUNG_DOMAIN: 15, YOU
NG_DOMAIN_DAYS: 30,
         NO_HTTPS: 5, SUSPICIOUS_KEYWORDS: 10, IP_ADDRESS_URL: 10, EXCESSIVE_SUBDOMAIN
S: 5, AT_SYMBOL_OR_PUNYCODE: 10 },
  file: { VT_PER_ENGINE: 8, VT_MAX: 100 },
  ip: { ABUSE_CONFIDENCE_MULTIPLIER: 1, MANY_REPORTS_BONUS: 10, MANY_REPORTS: 20 },
  message: { LINK_SCORE_WEIGHT: 0.5 },
};
export const THRESHOLDS = { SAFE_MAX: 30, SUSPICIOUS_MAX: 60 };
export function verdictFromScore(score) {
  if (score <= THRESHOLDS.SAFE_MAX) return 'safe';
  if (score <= THRESHOLDS.SUSPICIOUS_MAX) return 'suspicious';
  return 'dangerous';
}
export function scoreUrl({ safeBrowsing, virusTotal, domainAge, heuristics }) {
  const W = WEIGHTS.url; const breakdown = []; let score = 0;
  if (safeBrowsing?.available && safeBrowsing.matched) {
    score += W.SAFE_BROWSING_MATCH;
    breakdown.push({ label: `Google Safe Browsing: ${safeBrowsing.threats.join(', ')}`
, points: W.SAFE_BROWSING_MATCH });
  }
  if (virusTotal?.available && virusTotal.stats) {
    const flagged = (virusTotal.stats.malicious || 0) + (virusTotal.stats.suspicious |
| 0);
    if (flagged > 0) {
      const pts = Math.min(W.VT_MAX, flagged * W.VT_PER_ENGINE);
      score += pts; breakdown.push({ label: `${flagged} VirusTotal engines flagged thi
s URL`, points: pts });
    }
  }
  if (domainAge?.available && domainAge.ageDays < W.YOUNG_DOMAIN_DAYS) {
    score += W.YOUNG_DOMAIN; breakdown.push({ label: `Domain registered only ${domainA
ge.ageDays} days ago`, points: W.YOUNG_DOMAIN });
  }
  if (heuristics) {
    if (!heuristics.isHttps) { score += W.NO_HTTPS; breakdown.push({ label: 'Uses HTTP
 instead of HTTPS', points: W.NO_HTTPS }); }
    if (heuristics.keywords?.length) { score += W.SUSPICIOUS_KEYWORDS; breakdown.push(
{ label: `Suspicious keywords (${heuristics.keywords.slice(0, 3).join(', ')})`, points
: W.SUSPICIOUS_KEYWORDS }); }
    if (heuristics.isIpUrl) { score += W.IP_ADDRESS_URL; breakdown.push({ label: 'Raw
IP address used as the host', points: W.IP_ADDRESS_URL }); }
    if (heuristics.hasAtSymbol || heuristics.hasPunycode) { score += W.AT_SYMBOL_OR_PU
NYCODE; breakdown.push({ label: 'Look-alike or disguised host', points: W.AT_SYMBOL_OR
_PUNYCODE }); }
  }
  const finalScore = Math.max(0, Math.min(100, Math.round(score)));
  return { score: finalScore, verdict: verdictFromScore(finalScore), breakdown };
}
```

Link scanner orchestrator (server/services/urlScanner.js):

```
const settled = (p) => p.then((v) => v).catch((e) => ({ available: false, reason: 'err
or', error: e.message }));

export async function scanUrl(rawUrl, { skipCache = false, quick = false } = {}) {
  const u = normalizeUrl(rawUrl);
  if (!u) throw httpError(400, 'invalid_url', 'Please enter a valid http:// or https:/
/ link.');
  const url = u.href;
  const cacheKey = `url:${url}`;
  if (!skipCache) { const cached = cacheGet(cacheKey); if (cached) return { ...cached,
 cached: true }; }
```

```
  const heuristics = analyzeUrlHeuristics(u);
  const [safeBrowsing, virusTotal, domainAge] = await Promise.all([
    settled(checkSafeBrowsing(url)),
    settled(scanUrlWithVirusTotal(url, { quick })),
    heuristics.isIpUrl ? Promise.resolve({ available: false, reason: 'ip_url' }) : set
tled(getDomainAge(getRegistrableDomain(u.hostname))),
  ]);
  const { score, verdict, breakdown } = scoreUrl({ safeBrowsing, virusTotal, domainAge
, heuristics });
  const ai = quick ? { available: false, text: fallbackExplanation({ scanType: 'url',
verdict, score, breakdown }) }
                   : await explainFindings({ scanType: 'url', input: url, verdict, sco
re, breakdown });
  const report = { scanType: 'url', input: url, domain: u.hostname, threatScore: score
, verdict, explanation: ai.text,
                   aiAvailable: ai.available, breakdown, sources: { safeBrowsing, viru
sTotal, domainAge, heuristics },
                   unavailableSources: [...(!safeBrowsing.available ? ['Google Safe Br
owsing'] : []), ...(!virusTotal.available ? ['VirusTotal'] : [])],
                   scannedAt: new Date().toISOString(), cached: false };
  cacheSet(cacheKey, report, virusTotal.status === 'queued' ? 60 * 1000 : undefined);
  return report;
}
```

Google Safe Browsing (server/services/safeBrowsing.js):

```
const THREAT_TYPES = ['MALWARE', 'SOCIAL_ENGINEERING', 'UNWANTED_SOFTWARE', 'POTENTIAL
LY_HARMFUL_APPLICATION'];
export async function checkSafeBrowsing(url) {
  if (!hasKey('SAFE_BROWSING_API_KEY')) return { available: false, reason: 'not_config
ured', matched: false, threats: [] };
  try {
    const { data } = await axios.post(`${ENDPOINT}?key=${process.env.SAFE_BROWSING_API
_KEY}`, {
      client: { clientId: 'cybershield', clientVersion: '1.0.0' },
      threatInfo: { threatTypes: THREAT_TYPES, platformTypes: ['ANY_PLATFORM'], threat
EntryTypes: ['URL'], threatEntries: [{ url }] },
    }, { timeout: 8000 });
    const matches = data?.matches || [];
    const threats = [...new Set(matches.map((m) => PRETTY[m.threatType] || m.threatTyp
e))];
    return { available: true, matched: threats.length > 0, threats };
  } catch (err) {
    return { available: false, reason: 'error', matched: false, threats: [] };
  }
}
```

VirusTotal lookups (server/services/virusTotal.js):

```
export const vtUrlId = (url) => Buffer.from(url).toString('base64').replace(/\+/g, '-'
).replace(/\//g, '_').replace(/=+$/, '');

export async function scanUrlWithVirusTotal(url, { quick = false } = {}) {
  const vt = client(); const id = vtUrlId(url);
  try {
    const { data } = await vt.get(`/urls/${id}`);
    const known = fromAttributes(data.data.attributes, { permalink: `https://www.virus
total.com/gui/url/${id}` });
    if (known.stats) return known;
  } catch (err) { if (err.response?.status !== 404) return handleError(err, 'get-url')
; }
  const form = new URLSearchParams({ url });
  const { data } = await vt.post('/urls', form, { headers: { 'content-type': 'applicat
ion/x-www-form-urlencoded' } });
```

```
  if (quick) return { available: true, status: 'queued', analysisId: data.data.id, sta
ts: null, engines: [], submitted: true };
  return { ...(await pollAnalysis(data.data.id)), submitted: true };
}

export async function lookupFileHash(sha256) {
  try {
    const { data } = await client().get(`/files/${sha256}`);
    const attrs = data.data.attributes;
    return fromAttributes(attrs, { found: true, fileType: attrs.type_description || nu
ll,
      popularThreat: attrs.popular_threat_classification?.suggested_threat_label || nu
ll,
      families: (attrs.popular_threat_classification?.popular_threat_name || []).map((
t) => t.value) });
  } catch (err) {
    if (err.response?.status === 404) return { available: true, status: 'not_found', f
ound: false };
    return handleError(err, 'get-file');
  }
}
```

Hash-first file scanner (server/services/fileScanner.js):

```
export async function scanFile({ sha256, fileName, size, mimeType, buffer }) {
  if (!SHA256_RE.test(sha256 || '')) throw httpError(400, 'invalid_hash', 'A valid SHA
-256 hash is required.');
  const hash = sha256.toLowerCase();
  if (buffer && createHash('sha256').update(buffer).digest('hex') !== hash) {
    throw httpError(400, 'hash_mismatch', 'The uploaded file does not match its SHA-25
6 hash.');
  }
  let vt = await lookupFileHash(hash);
  if (vt.available && vt.status === 'not_found') {
    if (!buffer) return { needsUpload: true, sha256: hash };
    vt = await uploadFileToVirusTotal(buffer, fileName);
    buffer = null;
  }
  const { score, verdict, breakdown } = scoreFile({ virusTotal: vt });
  return { scanType: 'file', fileName, sha256: hash, size, threatScore: score, verdict
, breakdown, sources: { virusTotal: vt } };
}
```

Message scanner (server/services/messageScanner.js):

```
export async function scanMessage(rawText) {
  const text = String(rawText || '').trim();
  if (!text) throw httpError(400, 'missing_text', 'Paste the SMS or email text to chec
k.');
  if (text.length > MAX_CHARS) throw httpError(400, 'text_too_long', `Message is too l
ong (max ${MAX_CHARS} characters).`);
  const urls = extractUrls(text).slice(0, MAX_LINKS);
  const [aiRaw, linkReports] = await Promise.all([
    analyzeMessage(text),
    Promise.all(urls.map((u) => scanUrl(u, { quick: true }).catch(() => null))),
  ]);
  const rules = analyzeMessageHeuristics(text);
  const ai = aiRaw.available ? { ...aiRaw, redFlags: [...aiRaw.redFlags], source: 'gem
ini' } : rules;
  if (aiRaw.available) {
    const seen = new Set(ai.redFlags.map((f) => f.phrase.toLowerCase()));
    for (const f of rules.redFlags) if (!seen.has(f.phrase.toLowerCase())) ai.redFlags
.push(f);
  }
```

```
  const links = linkReports.filter(Boolean).map((r) => ({ url: r.input, threatScore: r
.threatScore, verdict: r.verdict, summary: r.summary }));
  const { score, verdict, breakdown } = scoreMessage({ ai, rules, linkScans: links });
  return { scanType: 'message', input: text, threatScore: score, verdict, explanation:
 ai.explanation, advice: ai.advice,
           breakdown, sources: { ai, rules, links }, scannedAt: new Date().toISOString
() };
}
```

Scam rule engine (server/utils/messageHeuristics.js):

```
const PATTERNS = [
  { re: /\b(kyc|know your customer)\b/i, pts: 20, type: 'KYC fraud', reason: 'Banks ne
ver ask you to "update KYC" through a link in a message.' },
  { re: /(?<!\b(?:not|never|don'?t|mat|nahi|kabhi)\s)\b(share|send|tell|provide|enter|
batayein|bhejein)\b[^.]{0,40}\b(otp|one[- ]time password|pin)\b/i, pts: 20, type: 'OTP
 scam', reason: 'No bank or office will ever ask for your OTP or PIN.' },
  { re: /\b(won|winner|lottery|lucky draw|prize|jackpot)\b/i, pts: 20, type: 'Lottery
/ prize scam', reason: 'You cannot win a lottery you never entered.' },
  { re: /\b(scan|qr code)\b.{0,30}\b(receive|get|collect)\b/i, pts: 25, type: 'QR "rec
eive money" scam', reason: 'You NEVER need to scan a QR code to receive money.' },
  { re: /\b(cyber ?cell|cbi|narcotics|customs|police)\b[\s\S]{0,120}\b(parcel|aadhaar|
drugs|arrest|warrant)\b/i, pts: 30, type: 'Digital arrest', reason: 'Police never cont
act you over video about parcels - this is the digital-arrest scam.' },
  { re: /\b(band|block|bandh)\s+(ho|kar)\s*(jayega|jayegi)\b/i, pts: 20, type: 'Accoun
t-block threat', reason: '"Band ho jayega" is a pressure tactic.' },
];
export function analyzeMessageHeuristics(text) {
  const redFlags = []; const types = new Map(); let score = 0;
  for (const p of PATTERNS) {
    const m = text.match(p.re); if (!m) continue;
    score += p.pts; redFlags.push({ phrase: m[0].trim(), reason: p.reason });
    types.set(p.type, (types.get(p.type) || 0) + p.pts);
  }
  const riskScore = Math.min(100, score);
  const scamType = [...types.entries()].sort((a, b) => b[1] - a[1])[0]?.[0] || 'none';
  return { available: true, source: 'rules', riskScore, scamType, redFlags, verdict: r
iskScore <= 30 ? 'safe' : riskScore <= 60 ? 'suspicious' : 'scam' };
}
```

Gemini analysis with model fallback (server/services/gemini.js):

```
const FALLBACK_MODELS = ['gemini-3.5-flash-lite', 'gemini-3.1-flash-lite', 'gemini-3.6
-flash', 'gemini-3.8-flash'];
async function generate(params, preferredModel) {
  for (const m of candidates(preferredModel)) {
    try { return await ai().models.generateContent({ ...params, model: m }); }
    catch (err) {
      const msg = err.message || '';
      if (/404|NOT_FOUND/i.test(msg)) { disabledUntil.set(m, Infinity); continue; }
      if (/429|quota/i.test(msg)) { disabledUntil.set(m, Date.now() + QUOTA_COOLDOWN_M
S); continue; }
      if (/503|504|timeout/i.test(msg)) { disabledUntil.set(m, Date.now() + OUTAGE_COO
LDOWN_MS); continue; }
      throw err;
    }
  }
  throw new Error('No Gemini model available');
}
export async function analyzeMessage(text) {
  const res = await withCap(generate({ contents: text, config: { systemInstruction: ME
SSAGE_SYSTEM_PROMPT,
      responseMimeType: 'application/json', temperature: 0.2, maxOutputTokens: 2048, t
hinkingConfig: { thinkingBudget: 0 } } }), ANALYZE_CAP_MS);
```

```
  if (!res) return { available: false, reason: 'timeout' };
  const json = JSON.parse((res.text || '').trim());
  return { available: true, verdict: json.verdict, riskScore: Math.max(0, Math.min(100
, Number(json.riskScore) || 0)),
           scamType: String(json.scamType || 'none'), redFlags: (json.redFlags || []).
slice(0, 10),
           explanation: String(json.explanation || ''), advice: String(json.advice ||
'') };
}
```

IP checker (server/services/ipScanner.js):

```
export function isNonPublicIp(ip, version = isIP(ip)) {
  if (version === 4) return PRIVATE_V4.test(ip);
  if (version === 6) return /^(::(?:1)?$|fc|fd|fe[89ab]|ff|2001:db8:)/i.test(ip);
  return false;
}
export async function scanIp(rawIp) {
  let ip = String(rawIp || '').trim().replace(/^\[|\]$/g, '');
  const mapped = ip.match(/^::ffff:(\d{1,3}(?:\.\d{1,3}){3})$/i);
  if (mapped) ip = mapped[1];
  const version = isIP(ip);
  if (!version) throw httpError(400, 'invalid_ip', 'Enter a valid IPv4 (e.g. 8.8.8.8)
or IPv6 address.');
  if (isNonPublicIp(ip, version)) return { scanType: 'ip', input: ip, threatScore: nul
l, verdict: 'unknown', isPrivate: true,
      summary: 'This is a local or reserved address. Public reputation checks do not a
pply.' };
  const [abuseIpdb, geo] = await Promise.all([checkAbuseIpdb(ip), geoLookup(ip)]);
  const { score, verdict, breakdown } = scoreIp({ abuseIpdb });
  return { scanType: 'ip', input: ip, threatScore: score, verdict, breakdown, sources:
 { abuseIpdb, geo } };
}
```

Browser hashing and k-anonymity password check (client/src/lib):

```
const toHex = (buf) => [...new Uint8Array(buf)].map((b) => b.toString(16).padStart(2,
'0')).join('');
export async function sha256Hex(fileOrBlob) {
  const digest = await crypto.subtle.digest('SHA-256', await fileOrBlob.arrayBuffer())
;
  return toHex(digest);
}
export async function sha1HexUpper(text) {
  const digest = await crypto.subtle.digest('SHA-1', new TextEncoder().encode(text));
  return toHex(digest).toUpperCase();
}
export async function checkPwned(password, signal) {
  const hash = await sha1HexUpper(password);
  const prefix = hash.slice(0, 5), suffix = hash.slice(5);
  const res = await fetch(`https://api.pwnedpasswords.com/range/${prefix}`, { headers:
 { 'Add-Padding': 'true' }, signal });
  let breachCount = 0, candidates = 0;
  for (const line of (await res.text()).split('\n')) {
    const [sfx, count] = line.trim().split(':'); const n = Number(count) || 0;
    if (n > 0) candidates++;
    if (sfx === suffix) breachCount = n;
  }
  return { available: true, prefix, breachCount, candidates };
}
```

QR parsing and UPI scoring (client/src/lib/qr.js, localScore.js):

```
export function parseQrContent(raw) {
  const text = String(raw || '').trim();
  if (/^(upi|gpay|paytmmp|phonepe|bhim|tez):\/\//i.test(text)) {
    const p = new URLSearchParams(text.slice(text.indexOf('?') + 1));
    const amount = p.get('am') ? Number(p.get('am')) : null;
    return { kind: 'upi', raw: text, upi: { payeeAddress: p.get('pa') || '', payeeName
: p.get('pn') || '',
             amount: Number.isFinite(amount) ? amount : null, merchantCode: p.get('mc'
) || '', note: p.get('tn') || '' } };
  }
  if (/^https?:\/\//i.test(text) || /^www\./i.test(text)) return { kind: 'url', raw: t
ext, url: text };
  if (/^WIFI:/i.test(text)) return { kind: 'wifi', raw: text };
  if (/^tel:/i.test(text)) return { kind: 'tel', raw: text, phone: text.slice(4) };
  return { kind: 'text', raw: text };
}
export function scoreUpi(upi) {
  const W = LOCAL_WEIGHTS.upi; const breakdown = []; let score = W.PAYMENT_REQUEST;
  breakdown.push({ label: 'QR is a UPI payment request (scanning it PAYS someone)', po
ints: W.PAYMENT_REQUEST });
  if (upi.amount > 0) { score += W.FIXED_AMOUNT; breakdown.push({ label: `Pre-filled a
mount Rs ${upi.amount}`, points: W.FIXED_AMOUNT }); }
  const handle = (upi.payeeAddress.split('@')[1] || '').toLowerCase();
  if (!KNOWN_HANDLES.includes(handle)) { score += W.UNKNOWN_HANDLE; breakdown.push({ l
abel: `Unusual UPI handle "@${handle}"`, points: W.UNKNOWN_HANDLE }); }
  if (SCAM_WORDS.test(`${upi.payeeName} ${upi.note}`)) { score += W.SCAM_KEYWORDS; bre
akdown.push({ label: 'Scam keywords in payee name / note', points: W.SCAM_KEYWORDS });
 }
  const s = Math.min(100, score); return { score: s, verdict: verdictFromScore(s), bre
akdown };
}
```

Scanner page scan runner (client/src/pages/Scanner.jsx):

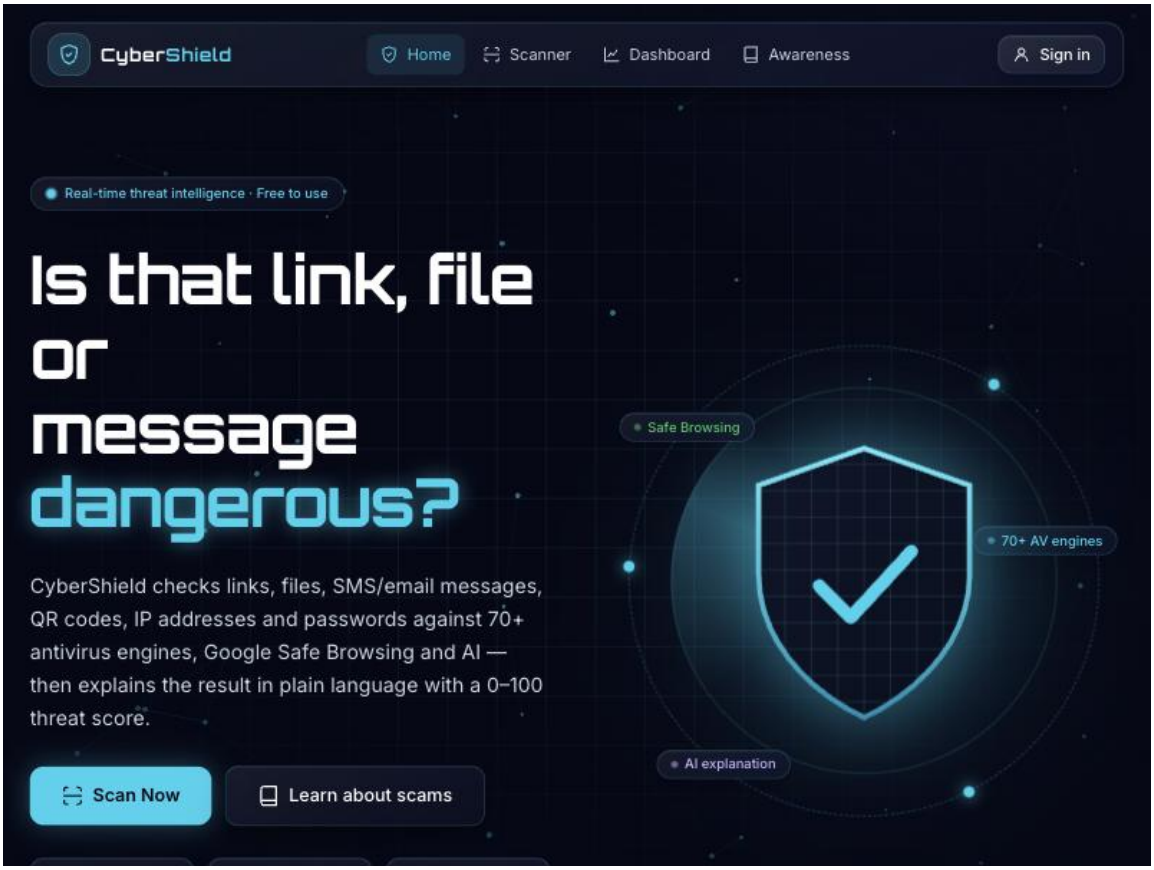
```
const runScan = async (fn, { lines = STATUS_LINES[type], preview = '' } = {}) => {
  setError(null); setStatusLines(lines); setSubtitle(preview); setScanning(true);
  abortRef.current = new AbortController();
  try {
    const [result] = await Promise.all([fn(abortRef.current.signal), wait(MIN_SCAN_MS)
]);
    saveLastResult(result);
    bumpGlobalCounter();
    navigate('/result', { state: { result } });
  } catch (err) {
    if (err.name !== 'AbortError') setError({ type, value: err });
  } finally { setScanning(false); }
};
const handleFileScan = (file) => runScan(async (signal) => {
  const sha256 = await sha256Hex(file);
  const meta = { sha256, fileName: file.name, size: file.size, mimeType: file.type };
  const first = await api.scanFile(JSON.stringify(meta), { signal, headers: { 'content
-type': 'application/json' } });
  if (!first.needsUpload) return first;
  const form = new FormData(); form.append('file', file, file.name);
  Object.entries(meta).forEach(([k, v]) => form.append(k, String(v)));
  return api.scanFile(form, { signal });
}, { preview: file.name });
```

Firestore security rules (firestore.rules):

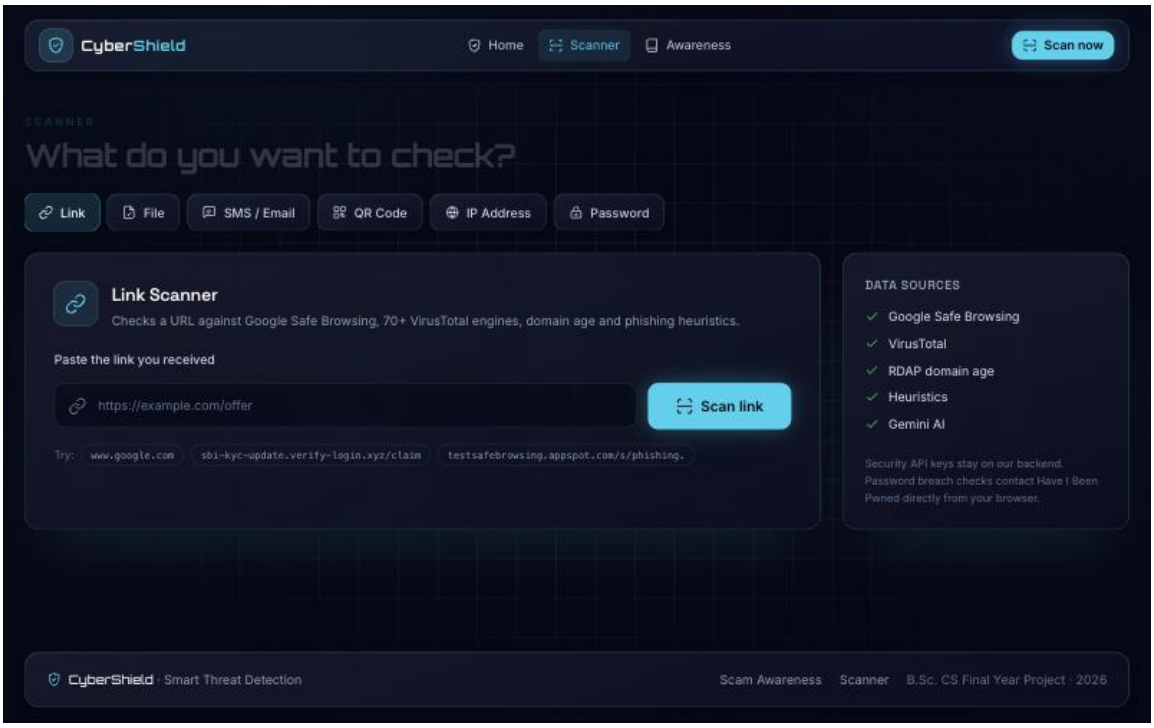
```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /stats/global {
      allow read: if true;
```

```
      allow create: if request.resource.data.keys().hasOnly(['totalScans']) && request
.resource.data.totalScans == 1;
      allow update: if request.resource.data.keys().hasOnly(['totalScans'])
                    && request.resource.data.totalScans == resource.data.totalScans +
1;
      allow delete: if false;
    }
    match /{document=**} { allow read, write: if false; }
  }
}
```

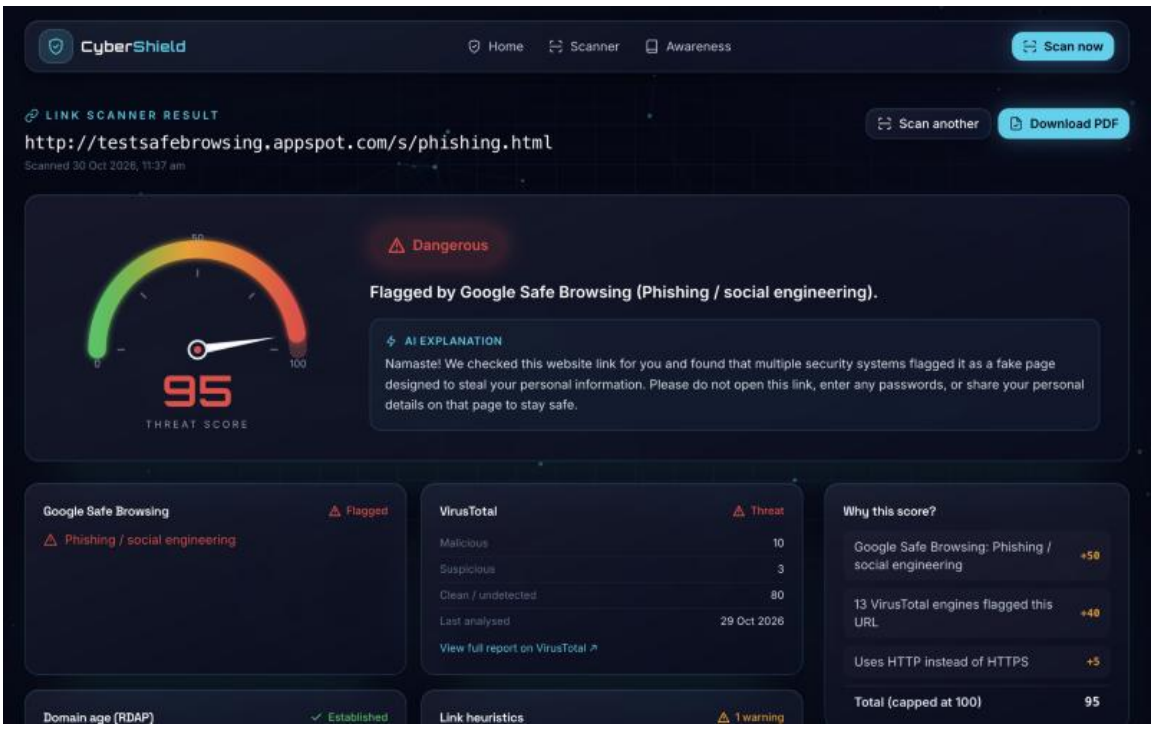
SCREEN LAYOUT



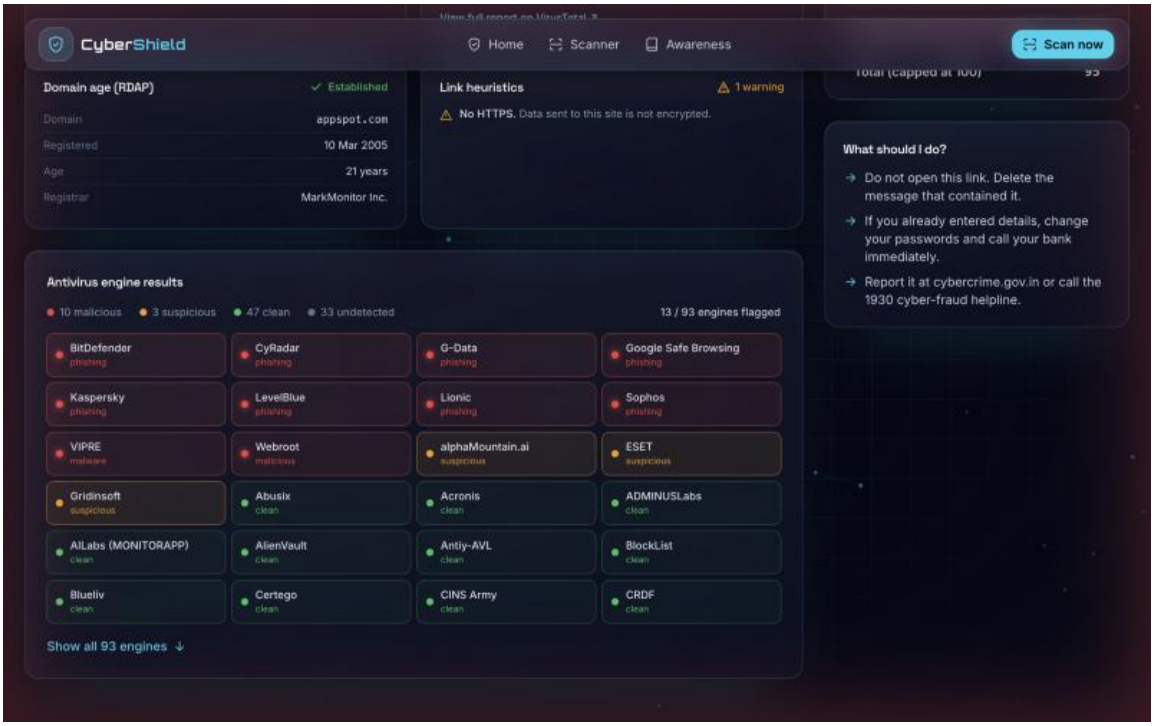
Home page with the six scanner cards and live scan counter



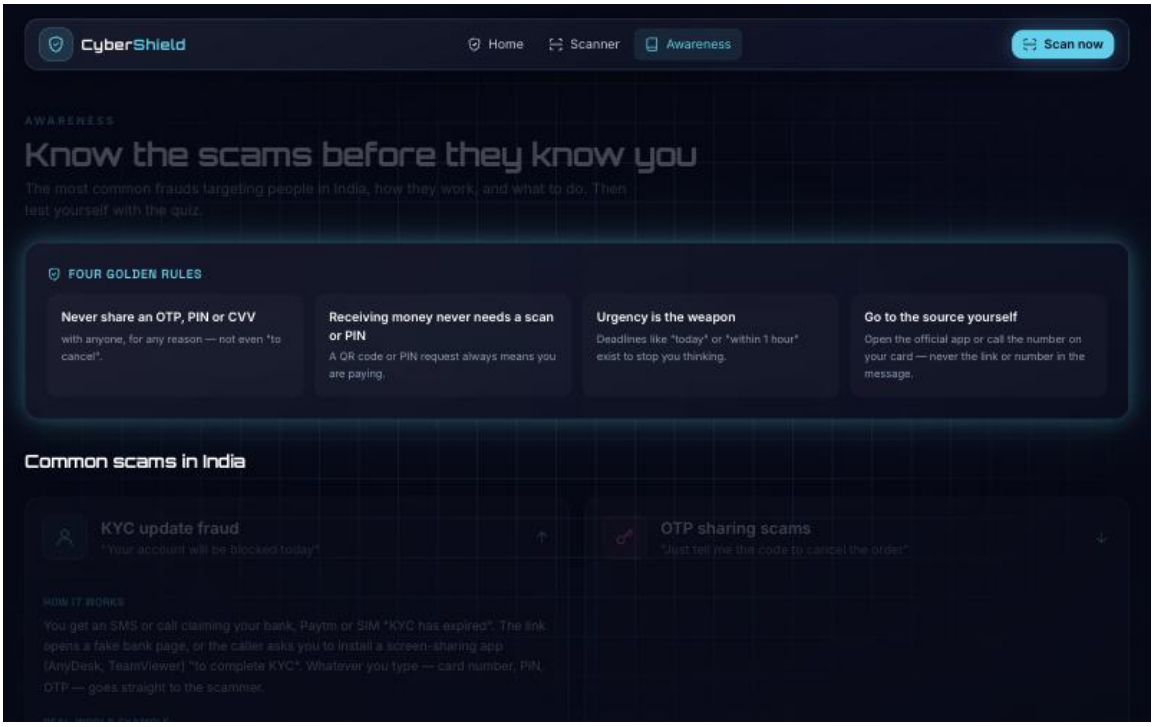
Scanner page with the Link tab selected



Result page: threat gauge, verdict badge and AI explanation



Result page: antivirus engine grid and score breakdown



Scam awareness page

CONCLUSION AND FUTURE WORK

FUTURE ENHANCEMENTS

Being web software, CyberShield has a huge scope. The system not only carries out the checks faster but also more reliably than manual checking, and it saves users from financial loss. The software needs to be maintained from time to time, for example when a security API changes or a new scam pattern appears. There are various aspects where the software can be developed further:

- Optional user accounts with a personal dashboard of past scans, charts and trends.
- A Chrome browser extension that checks links automatically before the user clicks them.
- An Android application that reads SMS messages and QR codes directly on the phone.
- More threat sources such as PhishTank, URLhaus and OpenPhish, and WHOIS data for more domain extensions.
- Visual phishing detection by rendering a page in a sandbox and comparing it with known brand pages.
- Scam detection and explanations in Hindi, Marathi and other regional languages.
- One-click reporting of a confirmed scam to the national cybercrime portal.
- A more attractive GUI with more reports and statistics for the administrator.

CONCLUSION:

The ‘CyberShield’ project keeps track of all the information needed to judge whether a link, file, message, QR code, IP address or password is dangerous. It brings together Google Safe Browsing, VirusTotal, RDAP, AbuseIPDB, Have I Been Pwned and Gemini AI behind one simple interface and converts their answers into a weighted threat score, a clear verdict and a plain-language explanation.

The users of the system are ordinary citizens who do not need to register or understand security tools. The administrator is the developer, who configures the API keys on the server and deploys the client on GitHub Pages and the API on Render.

Whenever a link is submitted it is normalised, checked by heuristics, looked up on the security services in parallel and cached. Files are identified by a hash computed in the browser and uploaded only when unknown. Messages are analysed by AI and a rule engine that understands Indian scam patterns in English and Hinglish. QR codes are decoded in the browser and UPI requests are explained. Passwords are checked with k-anonymity so they never leave the browser.

The testing of the major modules showed that the implemented features work successfully: known phishing and malware test pages were flagged, a test malware file was detected by its hash, scam messages were classified with highlighted red flags, abusive IP addresses were rated and leaked passwords were found, while genuine websites and messages were rated Safe.

Overall, CyberShield demonstrates how professional threat intelligence services and privacy-preserving techniques can be combined into a single, user-friendly web application, and it provides a useful foundation for developing more advanced security tools in the future.

BIBLIOGRAPHY

- **FOR REACT AND VITE**

  react.dev

  vite.dev

- **FOR NODE.JS AND EXPRESS**

  nodejs.org/docs

  expressjs.com

- **FOR TAILWIND CSS AND FRAMER MOTION**

  tailwindcss.com

  motion.dev

- **FOR SECURITY APIS**

  developers.google.com/safe-browsing

  docs.virustotal.com

  docs.abuseipdb.com

  haveibeenpwned.com/API/v3

  ai.google.dev/gemini-api/docs

- **FOR FIREBASE**

  firebase.google.com/docs/firestore

- **FOR DEPLOYMENT**

  docs.github.com/pages

  render.com/docs

- **FOR CYBER FRAUD AWARENESS**

  cybercrime.gov.in

  sancharsaathi.gov.in

  rbi.org.in (BE(A)WARE booklet)

REFERENCE WEBSITES

Website	Category
safebrowsing.google.com	Security > URL reputation
virustotal.com	Security > Multi-engine scanning
abuseipdb.com	Security > IP reputation
haveibeenpwned.com	Security > Breach checking
rdap.org	Internet > Domain registration data
ip-api.com	Internet > Geolocation
aistudio.google.com	AI > Gemini API
github.com/laxmiyadav72335/cybershield	Project > Source code
laxmiyadav72335.github.io/cybershield	Project > Live website
cybercrime.gov.in	Government > Report cyber fraud